



Programa Certified AI-LLM Integration Engineer

Curso de Fundamentos de LLMs y APIs

www.bsginstitute.com

Andrés Felipe Rojas Parra

- Ingeniero electrónico con mas de 23 años de experiencia
- PGP Artificial Intelligence & Machine Learning at Texas University – Austin, TX
- Master en Inteligencia Artificial y Big Data – IEBSchool
- Estudiante de PGP en Cloud Computing at Texas University - Austin, TX
- Texas Executive Education Program – McCombs School of Business – Decision Science and AI Immersion Program
- Estudiante de Especialización en MLOps de la Universidad de Duke

 andres.rojas@triskelss.com / arojaspa@outlook.com / arojaspa76

 @arojaspa

 <https://www.linkedin.com/in/arojaspa/>

 +573005906373



Agenda

1. Unidad 1 y 2

- Introducción a Deep learning
- Frameworks – ejercicios
 - ✓ Métodos de dropout & batch normalization
 - ✓ Fine tuning hyperparameters
 - ✓ Audio recognition
- ANN & CNN

2. Unidad 3 y 4

- Introducción a Computer Vision
- Preentrenamiento & Transferencia de aprendizaje
- Introducción al procesamiento de lenguaje natural (NLP)
- Análisis de sentimientos y datos de textos

Introducción a Deep learning

CONFIDENCIAL

Introducción a Deep learning

Objetivo del curso:

Después de completar este curso, usted podrá:

1. Comprender las imágenes como datos y los pasos para trabajar con ellos.
2. Comprender las diferencias entre las redes neuronales artificiales y las redes neuronales convolucionales.
3. Comprender y crear redes neuronales convolucionales para tareas de clasificación.

Que veremos en este módulo:

1. Introducción a Deep Learning y a la visión computacional
 - Diferencia entre ANN y CNN
 - La arquitectura de CNN
 - Ejercicio: Clasificación de dígitos escritos a mano
2. Regularización y aprendizaje por transferencia
 - Regularización de CNN
 - Introducción al aprendizaje por transferencia
 - Introducción a las arquitecturas clásicas del aprendizaje por transferencia
 - Ejercicio: Aprendizaje por transferencia en CNN

Introducción a Deep learning

Deep Learning, siendo un subconjunto de Machine Learning basado en redes neuronales artificiales con aprendizaje de representación, ha tenido un impacto significativo en numerosos dominios científicos e industriales. En comparación con los métodos tradicionales de Machine Learning, sus ventajas y desventajas tienen matices y varían según el contexto de la aplicación.

Ventajas del Deep Learning:

- **Extracción automática de características (features):** a diferencia de Machine Learning tradicional, los algoritmos de Deep Learning descubren automáticamente las representaciones necesarias para la detección o clasificación de características (features) a partir de datos sin procesar. Esto reduce la necesidad de realizar procedimientos manuales para hallar características (features) que se encuentran dentro conjunto de datos.
- **Manejo de datos de grandes dimensiones:** Deep Learning es particularmente adecuado para manejar datos de altas dimensiones (como imágenes y videos), gracias a su capacidad para aprender representaciones de características jerárquicas.
- **Rendimiento superior:** para problemas complejos, los modelos de Deep Learning a menudo logran métricas de rendimiento superiores, superando a veces las capacidades de nivel humano en tareas como el reconocimiento de imágenes, el procesamiento del lenguaje natural y los juegos.

Introducción a Deep learning

- **Flexibilidad:** los modelos de Deep Learning se pueden adaptar y aplicar a una amplia variedad de problemas y dominios, desde visión por computadora hasta análisis de datos secuenciales, como series de tiempo.
- **Integración con Big Data:** Estos modelos pueden mejorar a medida que consumen más datos, lo que los hace ideales para aprovechar las grandes cantidades de datos generados en la era digital.

Desventajas del Deep Learning:

- **Requisitos de datos:** los modelos de Deep Learning suelen requerir grandes cantidades de datos de entrenamiento etiquetados para lograr un alto rendimiento. Recopilar y etiquetar estos datos puede resultar costoso y llevar mucho tiempo.
- **Recursos computacionales:** el entrenamiento de modelos de Deep Learning a menudo requiere recursos computacionales sustanciales (por ejemplo, GPU, TPU), lo que lo hace menos accesible para personas u organizaciones con hardware limitado.
- **Interpretabilidad del modelo:** los modelos de Deep Learning a menudo se consideran "cajas negras", lo que significa que puede resultar difícil comprender cómo toman decisiones o predicciones, lo que puede ser un inconveniente crítico en ámbitos sensibles como la atención sanitaria o las finanzas.

Introducción a Deep learning

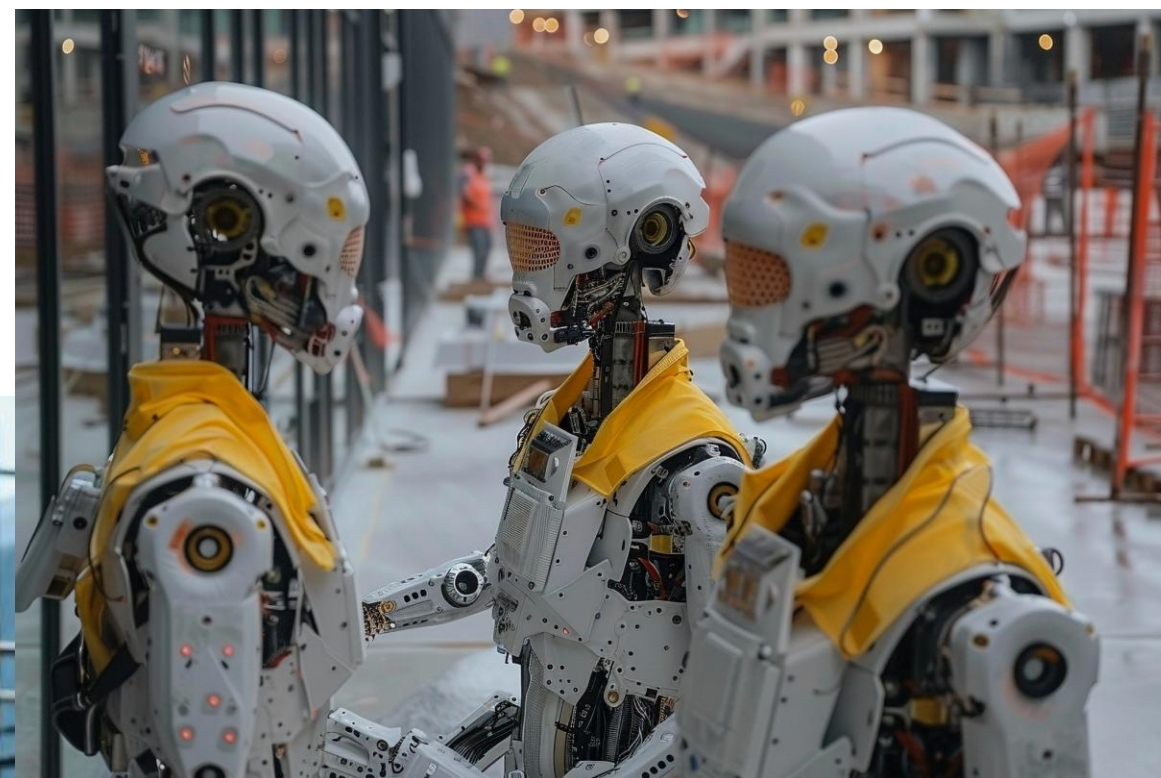
- **Riesgo de Overfitting:** Dada su complejidad y capacidad de memorización, existe riesgo de overfitting, especialmente cuando se entrenan con datos limitados o sin técnicas de regularización adecuadas.
- **Conocimiento y experiencia:** diseñar, entrenar e implementar modelos de Deep Learning de manera efectiva requiere conocimientos especializados en redes neuronales, optimización y el dominio de la aplicación en la que se va a usar el modelo.
- **Preprocesamiento:** aunque Deep Learning es necesario para extraer características de aprendizaje, aún se necesitan tareas de preprocesamiento como la normalización, el manejo de valores faltantes y la conversión de datos a un formato adecuado para redes neuronales.
- **Conocimiento del dominio de la aplicación:** si bien es un problema menor en comparación con las técnicas tradicionales, el conocimiento del dominio sigue siendo crucial para definir el problema, diseñar la arquitectura de la red e interpretar los resultados.

En resumen, Deep Learning es una herramienta poderosa para resolver problemas complejos mediante el aprendizaje automático de patrones intrincados en grandes conjuntos de datos. Sin embargo, su eficacia conlleva salvedades con respecto a los datos, los recursos computacionales, la interpretabilidad y la experiencia. Equilibrar estos factores es fundamental para aprovechar el Deep Learning y al mismo tiempo superar sus limitaciones.

**¿Qué es Deep
learning?**

CONFIDENTIAL

!!!Esto no es Deep Learning!!!



Artificial Intelligence (IA) - Inteligencia Artificial.

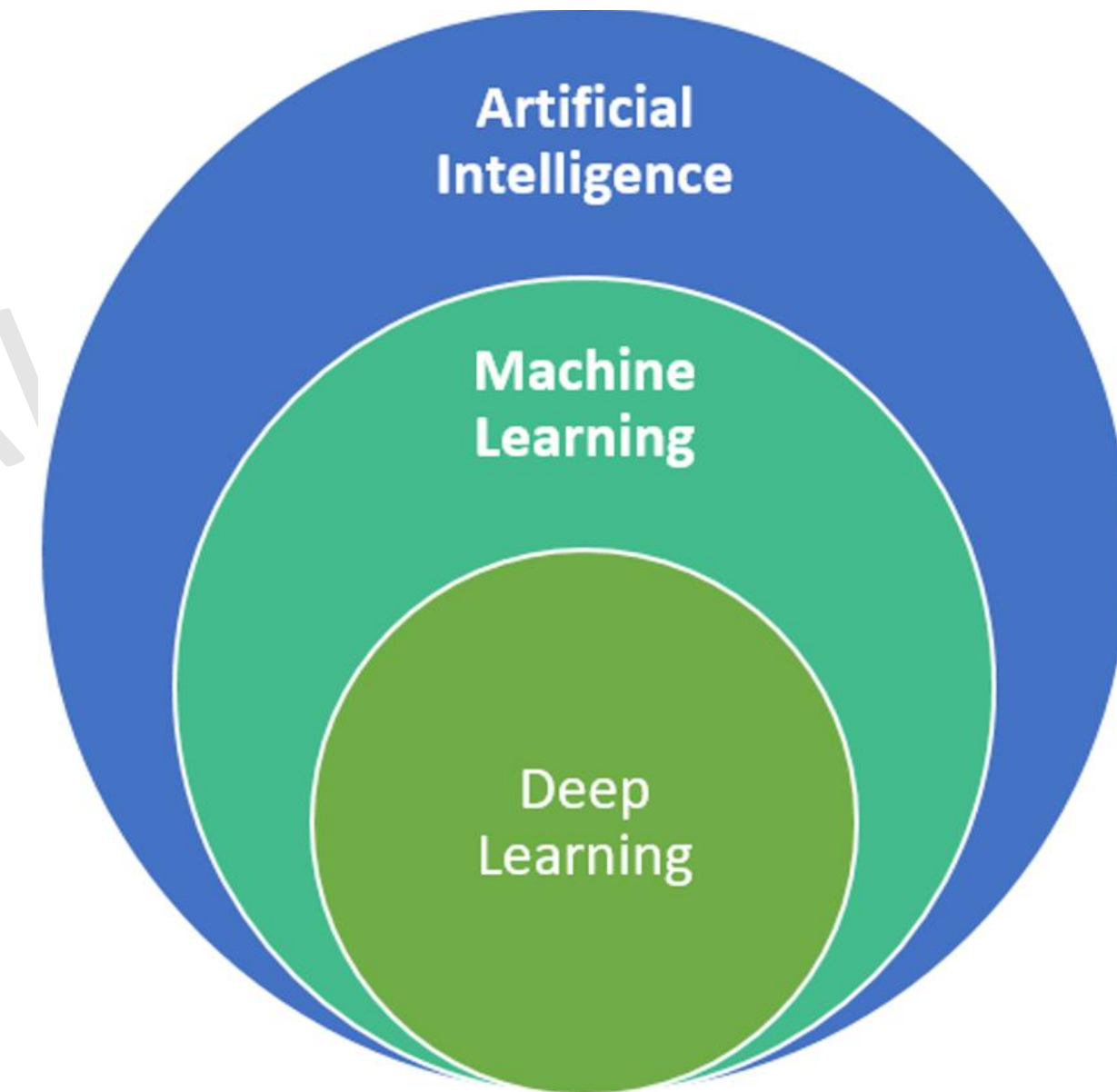
Es un campo de la informática que tiene como objetivo crear sistemas capaces de realizar tareas que normalmente requieren inteligencia humana. Estas tareas incluyen cosas como el reconocimiento de voz, la toma de decisiones, la comprensión del lenguaje natural y la visión por computadora. La IA abarca una gran variedad de técnicas, incluidas las basadas en reglas, los algoritmos heurísticos, los sistemas expertos, y, por supuesto, el aprendizaje automático (Machine Learning). Un ejemplo común es un **asistente virtual** (como Siri o Alexa), que combina varias capacidades de IA para comprender comandos de voz, buscar información y realizar acciones.

Machine Learning (ML) - Aprendizaje Automático

Es una subdisciplina dentro de la Inteligencia Artificial que se enfoca en desarrollar algoritmos y modelos que permitan a las máquinas aprender de los datos. En lugar de programar explícitamente todas las reglas, los modelos de Machine Learning aprenden patrones a partir de grandes conjuntos de datos. ML es un subconjunto de la IA, y dentro de ML se incluyen enfoques como **la regresión, la clasificación, el clustering y el aprendizaje supervisado y no supervisado**. Algunos ejemplos incluyen los **sistemas de recomendación** que sugieren productos o películas (como Netflix o Amazon), y los **motores de búsqueda** que ajustan los resultados en función de las preferencias de los usuarios.

Deep Learning (DL) - Aprendizaje Profundo

Es un subconjunto de Machine Learning que utiliza **redes neuronales artificiales profundas** para modelar patrones complejos en grandes cantidades de datos. Lo que diferencia a Deep Learning es la profundidad de las redes (capas ocultas múltiples), que permiten aprender características más abstractas y complejas. DL es una técnica dentro del aprendizaje automático, pero lo que lo hace especial, es su capacidad para manejar grandes cantidades de datos y aprender de ellos sin la necesidad de un preprocesamiento manual significativo. Requiere muchos recursos computacionales y grandes volúmenes de datos. Los **modelos de reconocimiento de imágenes** (como los usados en diagnósticos médicos por imágenes) y los **modelos de procesamiento del lenguaje natural** (como **GPT-3**, que puede generar texto coherente a partir de una instrucción dada) son ejemplos de deep learning.



¿Cuándo se inventó el Deep Learning?

El concepto de **Deep Learning** se remonta a las primeras investigaciones sobre **redes neuronales artificiales** en la década de 1940, cuando **Warren McCulloch** y **Walter Pitts** propusieron un modelo matemático de neuronas biológicas. Sin embargo, la implementación de redes neuronales profundas tal como las conocemos hoy no fue viable hasta mucho después debido a limitaciones computacionales.

Algunos hitos clave en la evolución del Deep Learning:

- 1. 1943:** McCulloch y Pitts propusieron el primer modelo de una neurona artificial.
- 2. 1957:** **Frank Rosenblatt** inventó el perceptrón, una red neuronal simple con capacidad de aprendizaje.
- 3. 1986:** **Geoffrey Hinton**, **David Rumelhart**, y **Ronald Williams** desarrollaron el algoritmo de **retropropagación del error** (backpropagation), que hizo posible entrenar redes neuronales multicapa, sentando las bases para lo que más tarde se llamaría Deep Learning.
- 4. 2006:** **Geoffrey Hinton**, junto con sus estudiantes **Yann LeCun** y **Yoshua Bengio**, reavivó el campo de las redes neuronales profundas al desarrollar un enfoque basado en el preentrenamiento de redes neuronales profundas utilizando capas de autoencoders. Este fue el punto en el que se popularizó el término **Deep Learning**.
- 5. 2012:** Un punto de inflexión ocurrió cuando una red neuronal profunda entrenada por el equipo de **Geoffrey Hinton** ganó la competencia de visión por computadora **ImageNet**, reduciendo dramáticamente los errores en el reconocimiento de imágenes. Este evento marcó el comienzo de la era moderna del Deep Learning.

Empresas y Capital Invertido en Deep Learning

Desde los años 2010, varias empresas han invertido cantidades significativas en investigación y desarrollo de **Deep Learning**. Aquí hay algunas de las más destacadas:

- **Google (Alphabet):**

- ❖ **Inversión:** Google ha sido uno de los mayores inversores en Deep Learning, especialmente a través de su subsidiaria **DeepMind**, que fue adquirida en 2014 por \$500 millones.
- ❖ **Proyectos clave:** **AlphaGo**, que derrotó al campeón mundial de Go en 2016, y el desarrollo de **TensorFlow**, una de las plataformas más populares para Deep Learning.
- ❖ **Google Brain:** Este grupo de investigación se centra en IA y aprendizaje automático, incluidos proyectos de deep learning.

- **Microsoft:**

- ❖ **Inversión:** Microsoft ha invertido significativamente en IA a través de su división Microsoft Research y en su plataforma de inteligencia artificial Azure AI. También ha invertido en empresas de IA, como su asociación con OpenAI, a la que ha inyectado más de \$1,000 millones.
- ❖ **Proyectos clave:** El uso de deep learning en su suite de productos como Cortana, Azure AI, y su integración en Microsoft Office.

- **Facebook (Meta):**

- ❖ **Inversión:** Facebook creó el grupo de investigación **FAIR (Facebook AI Research)**, que ha liderado el desarrollo de muchas aplicaciones basadas en deep learning.
- ❖ **Proyectos clave:** **PyTorch**, una plataforma de código abierto de deep learning muy utilizada, fue desarrollada por FAIR. También han utilizado deep learning en su tecnología de reconocimiento facial y sistemas de recomendación.

Empresas y Capital Invertido en Deep Learning

- **Amazon:**

- ❖ **Inversión:** A través de **AWS (Amazon Web Services)**, Amazon ha invertido millones de dólares en servicios de IA y deep learning en la nube. AWS ofrece herramientas como **SageMaker** para entrenar modelos de machine learning y deep learning.
- ❖ **Proyectos clave:** Amazon utiliza redes neuronales profundas para la recomendación de productos, Alexa, y sistemas de visión por computadora.

- **Nvidia:**

- ❖ **Inversión:** Nvidia, como fabricante de **GPUs**, ha sido fundamental en el desarrollo de hardware para deep learning. Ha invertido enormemente en investigación y desarrollo de algoritmos de deep learning optimizados para su hardware.
- ❖ **Proyectos clave:** Desarrollo de **CUDA**, una plataforma para computación paralela que es crucial para el entrenamiento de redes profundas. Nvidia también ha liderado la creación de arquitecturas especializadas como **NVIDIA DGX** para deep learning.

- **OpenAI:**

- ❖ **Inversión:** **OpenAI**, fundada con un objetivo inicial de ser una organización sin fines de lucro, ha recibido miles de millones en inversiones, principalmente de Microsoft. OpenAI ha liderado la creación de modelos de deep learning masivos como **GPT-3** y **DALL-E**.
- ❖ **Proyectos clave:** **GPT (Generative Pre-trained Transformer)**, un ejemplo de deep learning aplicado al procesamiento del lenguaje natural, que ha sido revolucionario en la generación de texto.

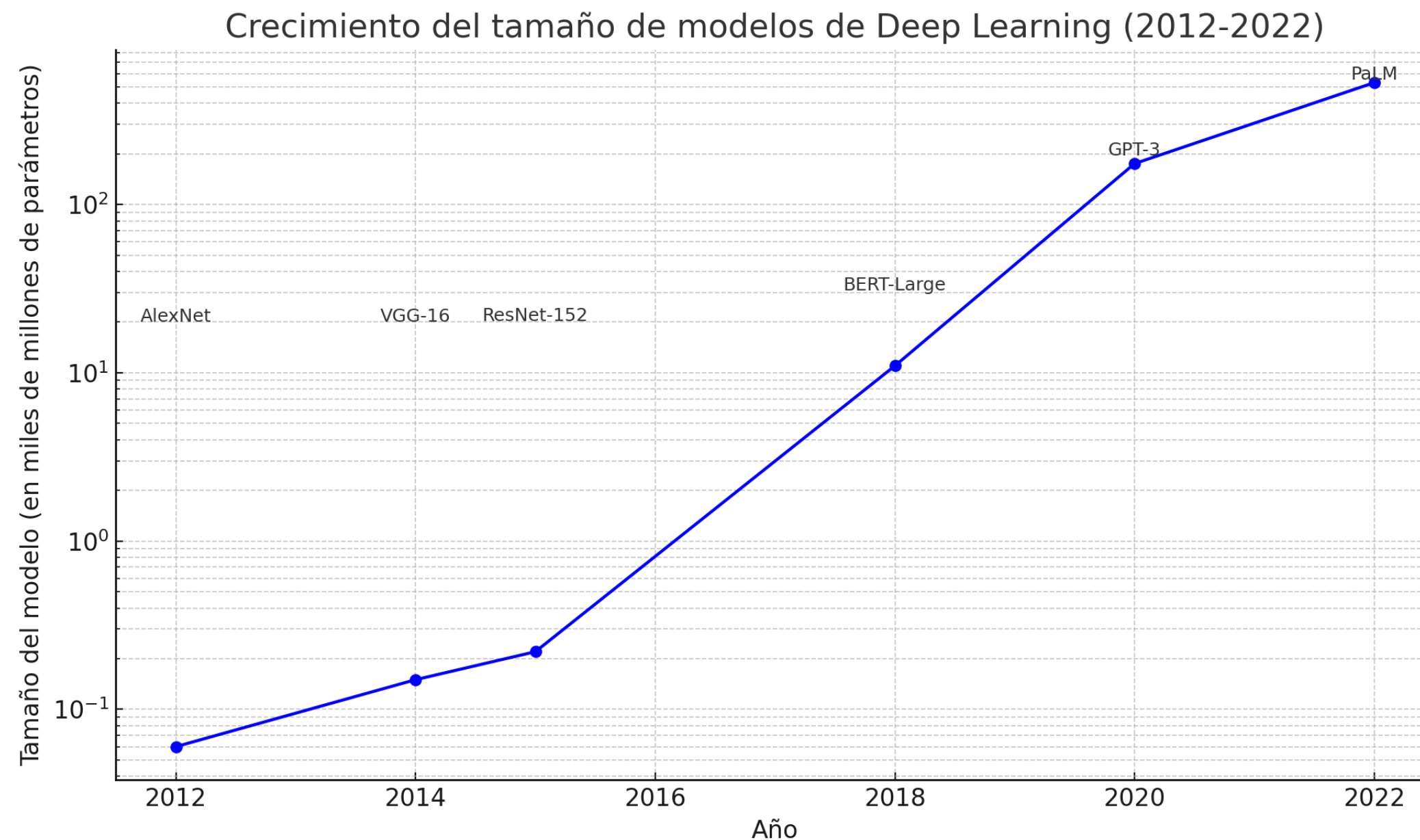
En términos de cuánto del total del gasto en investigación y desarrollo de **Inteligencia Artificial** corresponde a **Machine Learning (ML)** y **Deep Learning (DL)**, es difícil determinar porcentajes exactos, pero podemos tener una idea general basada en las siguientes observaciones:

1. **Deep Learning** ha crecido de manera exponencial en la última década gracias al aumento de la capacidad computacional (GPUs, TPUs) y la disponibilidad de grandes cantidades de datos.
2. **Machine Learning** sigue siendo un área más amplia, ya que no todas las aplicaciones de AI requieren deep learning. **Aprendizaje supervisado y no supervisado, regresiones, clustering**, y otros métodos más "clásicos" de Machine Learning siguen siendo altamente utilizados en muchas industrias.

Estimaciones:

1. Se estima que **Deep Learning** podría representar entre el **40% y el 60%** de la inversión total en **Machine Learning** dentro de **AI**, dependiendo del sector. Por ejemplo, en áreas como la **visión por computadora** y el **procesamiento del lenguaje natural (NLP)**, Deep Learning podría representar un porcentaje más alto (cerca del 60%-70%), mientras que, en otras áreas como la **modelización predictiva más tradicional** o los **sistemas de recomendación**, el Machine Learning clásico sigue siendo más predominante.
2. Dentro del total de la inversión en **AI**, **Machine Learning y Deep Learning juntos** podrían representar aproximadamente el **75%-85%** del gasto total en **I+D en AI**, con el resto dividido entre otros subcampos de AI como **IA basada en reglas** o **sistemas expertos**.

Deep Learning ha crecido significativamente desde sus primeros desarrollos teóricos y ha atraído enormes inversiones por parte de gigantes tecnológicos como Google, Microsoft, y Amazon. Si bien el **Machine Learning** abarca un conjunto más amplio de técnicas, el **Deep Learning** es hoy uno de los principales motores detrás de los avances más revolucionarios en inteligencia artificial, particularmente en áreas como visión artificial, procesamiento del lenguaje natural, y sistemas autónomos.



Este gráfico muestra el crecimiento exponencial del tamaño de los modelos de **Deep Learning** en términos de la cantidad de parámetros (en miles de millones) entre 2012 y 2022.

Evolución de los modelos más representativos:

1. **AlexNet (2012)**: El modelo pionero que marcó el inicio de la revolución del Deep Learning en visión por computadora, con aproximadamente **60 millones de parámetros**.
2. **VGG-16 (2014)**: Un modelo más profundo que alcanzó **150 millones de parámetros** y mejoró significativamente la precisión en tareas de clasificación de imágenes.
3. **ResNet-152 (2015)**: Con **220 millones de parámetros**, introdujo el concepto de **residual learning**, permitiendo entrenar redes mucho más profundas.
4. **BERT-Large (2018)**: Un modelo basado en **transformers** para procesamiento de lenguaje natural, con aproximadamente **11 mil millones de parámetros**.
5. **GPT-3 (2020)**: Uno de los modelos más avanzados para generación de lenguaje natural, con **175 mil millones de parámetros**.
6. **PaLM (2022)**: Modelo de Google con **530 mil millones de parámetros**, lo que lo convierte en uno de los modelos más grandes hasta la fecha.

Claves del crecimiento:

1. **Hardware especializado:** El desarrollo de hardware como **GPUs**, **TPUs** y arquitecturas especializadas ha permitido entrenar modelos con miles de millones de parámetros.
2. **Aumento de datos:** La disponibilidad de grandes cantidades de datos ha facilitado el entrenamiento de modelos más grandes y complejos.
3. **Mejoras en los algoritmos:** Técnicas como el **preentrenamiento**, la **regularización** y **optimizadores avanzados** han permitido que los modelos manejen de manera eficiente enormes cantidades de parámetros sin sobreajustar.

Deep Learning y LLM:

El **Deep Learning** ha aprovechado esta capacidad de cómputo avanzada para entrenar modelos con miles de millones de parámetros, lo que ha permitido que se logren avances significativos en áreas como la **visión por computadora**, **procesamiento del lenguaje natural** y **sistemas generativos**. Los modelos más recientes, como **GPT-3** y **PaLM**, han revolucionado tareas como la generación de texto, traducción automática, y creación de contenido sintético, tareas que antes eran consideradas extremadamente complejas.

Conocimiento de Software antes de la Evolución de la Capacidad de Cómputo

Comparación del Enfoque del Pasado vs. Presente:

Aspecto	Antes (pre-revolución del cómputo)	Hoy (con capacidad de cómputo avanzada)
Lenguajes comunes	FORTRAN, C/C++, MATLAB, R	Python, Julia, C++, JavaScript (TensorFlow.js)
Ingeniería de características	Manual, dependiente del conocimiento humano	Automática, mediante redes neuronales profundas
Modelos preferidos	Modelos simples (SVM, regresión, árboles de decisión)	Redes neuronales profundas, transformers
Escalabilidad	Limitada, algoritmos optimizados manualmente	Escalable, procesamiento paralelo en GPUs/TPUs
Entrenamiento	Pequeños datasets, largo tiempo de entrenamiento	Grandes datasets, optimización y entrenamiento masivo
Interfaz de desarrollo	Mayor complejidad en escribir código optimizado	API simples y de alto nivel (Keras, PyTorch)

La evolución de la capacidad de cómputo ha permitido que el **Deep Learning** y **Machine Learning** avancen enormemente.

Hoy en día, lenguajes como **Python** y bibliotecas como **TensorFlow** y **PyTorch** han hecho posible que incluso personas sin formación avanzada en optimización de hardware puedan construir modelos complejos con millones o miles de millones de parámetros.

Esto ha democratizado la inteligencia artificial, permitiendo que más personas puedan desarrollar aplicaciones avanzadas de IA sin la necesidad de optimización manual, como era necesario en décadas anteriores.

Evolución de la Capacidad de Cómputo

Resumen de la Influencia de las GPUs en Deep Learning:

Aspecto	Impacto de las GPUs
Procesamiento paralelo	Las GPUs tienen miles de núcleos para ejecutar cálculos paralelos en grandes cantidades de datos simultáneamente.
Aceleración del entrenamiento	El entrenamiento de redes neuronales profundas que tomaría semanas en CPU se puede reducir a horas o días con GPUs.
Compatibilidad con bibliotecas	Bibliotecas como TensorFlow, PyTorch y Keras están optimizadas para aprovechar las GPUs mediante frameworks como CUDA.
Escalabilidad	Las GPUs permiten entrenar modelos masivos como GPT-3 con miles de millones de parámetros.
Inferencia en tiempo real	Las GPUs aceleran la inferencia en aplicaciones como el reconocimiento facial y los vehículos autónomos.
GPU vs TPU	Las TPUs están optimizadas para Deep Learning, pero las GPUs son más versátiles y ampliamente utilizadas.

Las **TPUs** (desarrolladas por Google) son otra evolución tecnológica diseñada específicamente para acelerar el entrenamiento y la inferencia en modelos de Deep Learning. Mientras que las **GPUs** fueron originalmente diseñadas para procesamiento gráfico y se adaptaron para Deep Learning, las **TPUs** se desarrollaron desde cero con el único propósito de manejar cálculos relacionados con **Deep Learning**.

Diferencias: Las **TPUs** tienden a ser más rápidas y eficientes para tareas específicas de **tensor** y **matrices** en redes neuronales, pero las **GPUs** siguen siendo más versátiles y ampliamente adoptadas, ya que pueden manejar una mayor variedad de tareas, incluidos gráficos y cálculos más generales.

Las **GPUs** han sido fundamentales para el crecimiento y éxito del **Deep Learning** al permitir un procesamiento masivamente paralelo, lo que ha acelerado el entrenamiento y ha hecho posible la creación de modelos extremadamente grandes y complejos. Con el soporte continuo de las GPUs y tecnologías como **CUDA**, frameworks como **TensorFlow** y **PyTorch** se han optimizado para aprovechar al máximo su potencia, permitiendo a investigadores y empresas desarrollar soluciones de IA más avanzadas y eficientes.

Comparativa de extracción de características entre DL & ML

Comparación Directa ML vs DL en la Extracción de Características		
Aspecto	Machine Learning Tradicional (ML)	Deep Learning (DL)
Extracción de características	Manual y dependiente del conocimiento del dominio	Automática y aprendida durante el entrenamiento. Automática mediante aprendizaje de múltiples capas
Trabajo previo necesario	Alto: requiere limpieza, transformación, selección de características, etc.	Bajo: los modelos aprenden de los datos crudos
Eficiencia con datos no estructurados	Difícil: requiere preprocesamiento significativo	Excelente: las redes profundas aprenden de datos no estructurados directamente. Excelente para imágenes, texto, audio, etc.
Conocimiento del dominio	Esencial para la ingeniería de características	Menos necesario: las redes neuronales descubren características automáticamente
Facilidad de escalado	Difícil: el rendimiento se deteriora con características mal diseñadas o datos grandes	Fácil: los modelos de DL escalan bien con grandes volúmenes de datos
Dependencia del ingeniero de datos	Alta: se requiere conocimiento profundo del dominio	Baja: la red neuronal descubre las características automáticamente
Datos necesarios	Funciona bien con conjuntos de datos pequeños	Requiere grandes cantidades de datos
Capacidad jerárquica	No jerárquica: las características manuales pueden ser limitadas	Jerárquica: aprende características de bajo y alto nivel
Transparencia e interpretabilidad	Fácil de entender: las características son explícitas	Difícil de interpretar: las características aprendidas son abstractas
Ejemplo	Clasificación de spam basado en palabras clave predefinidas	Clasificación de imágenes sin necesidad de diseñar características de borde, textura, etc.

En **Machine Learning tradicional**, la **extracción de características** es **crítica** y requiere mucho trabajo manual y conocimiento del dominio. Este proceso consume tiempo y es uno de los aspectos más importantes para mejorar el rendimiento del modelo.

En **Deep Learning**, la **extracción de características** se realiza automáticamente a través de las capas de la red neuronal. Esto reduce significativamente el trabajo previo necesario y permite que los modelos sean altamente efectivos en tareas que involucran datos no estructurados como imágenes, texto y audio. Por lo tanto, DL requiere menos trabajo de preprocesamiento y extracción de características en comparación con ML tradicional, lo que lo hace más eficiente en problemas complejos.

Machine Learning tradicional necesita mucho más trabajo previo con los datos en términos de extracción de características, mientras que **Deep Learning** es más eficiente en ese aspecto debido a su capacidad para aprender características automáticamente de los datos.

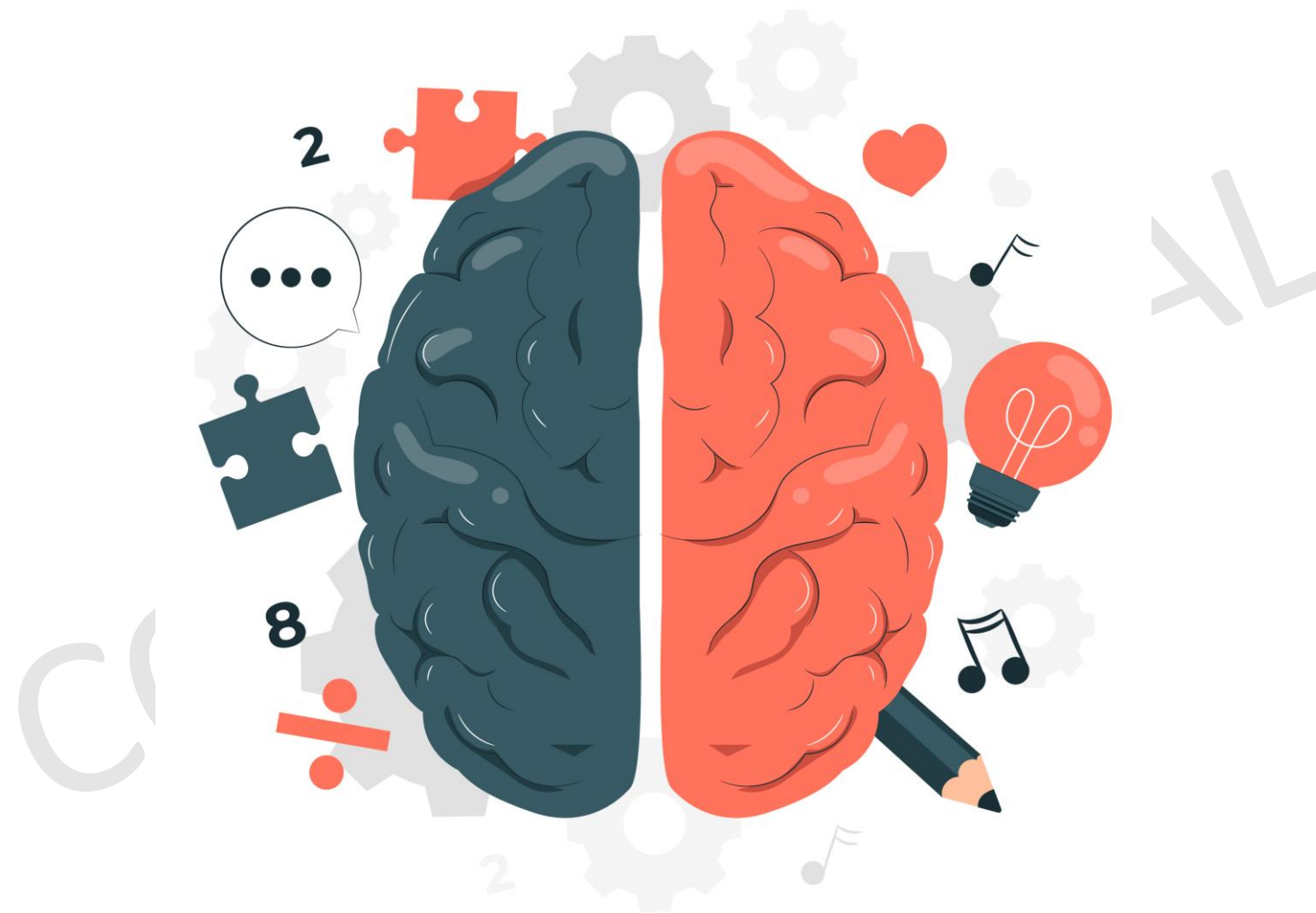
Redes Neuronales

CONFIDENCIAL

Cómo funciona el cerebro?

Nuestros sensores:

- Visión
- Tacto
- Olfato
- Gusto
- Escucha



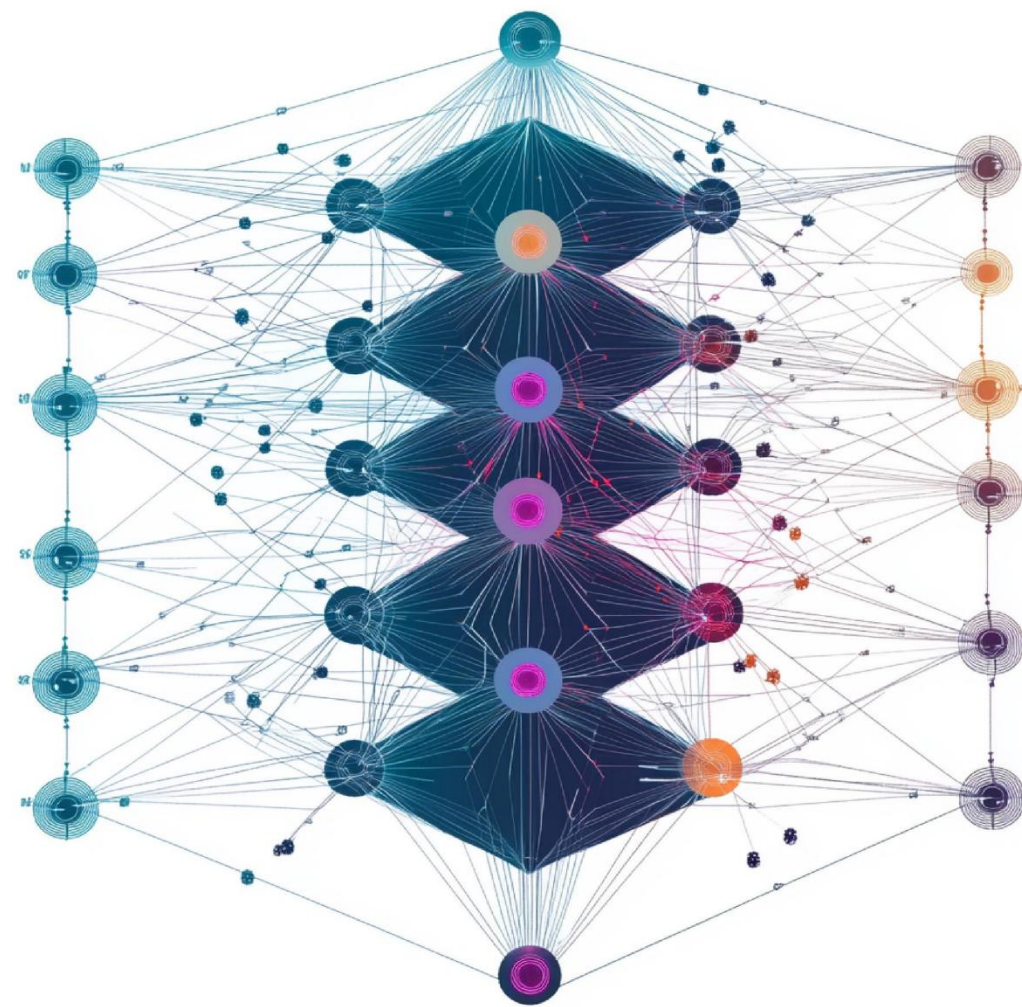
Que nos entregan:

- Poder identificar (clasificar) objetos
- Comparativas / Similitudes de objetos
- Mucha información de olores, sabores
- Patrones de comportamiento

Cómo funciona Deep Learning?

DL Sensores o
Fuentes de datos

- Texto
- Audio
- Video
- Imágenes
- Números

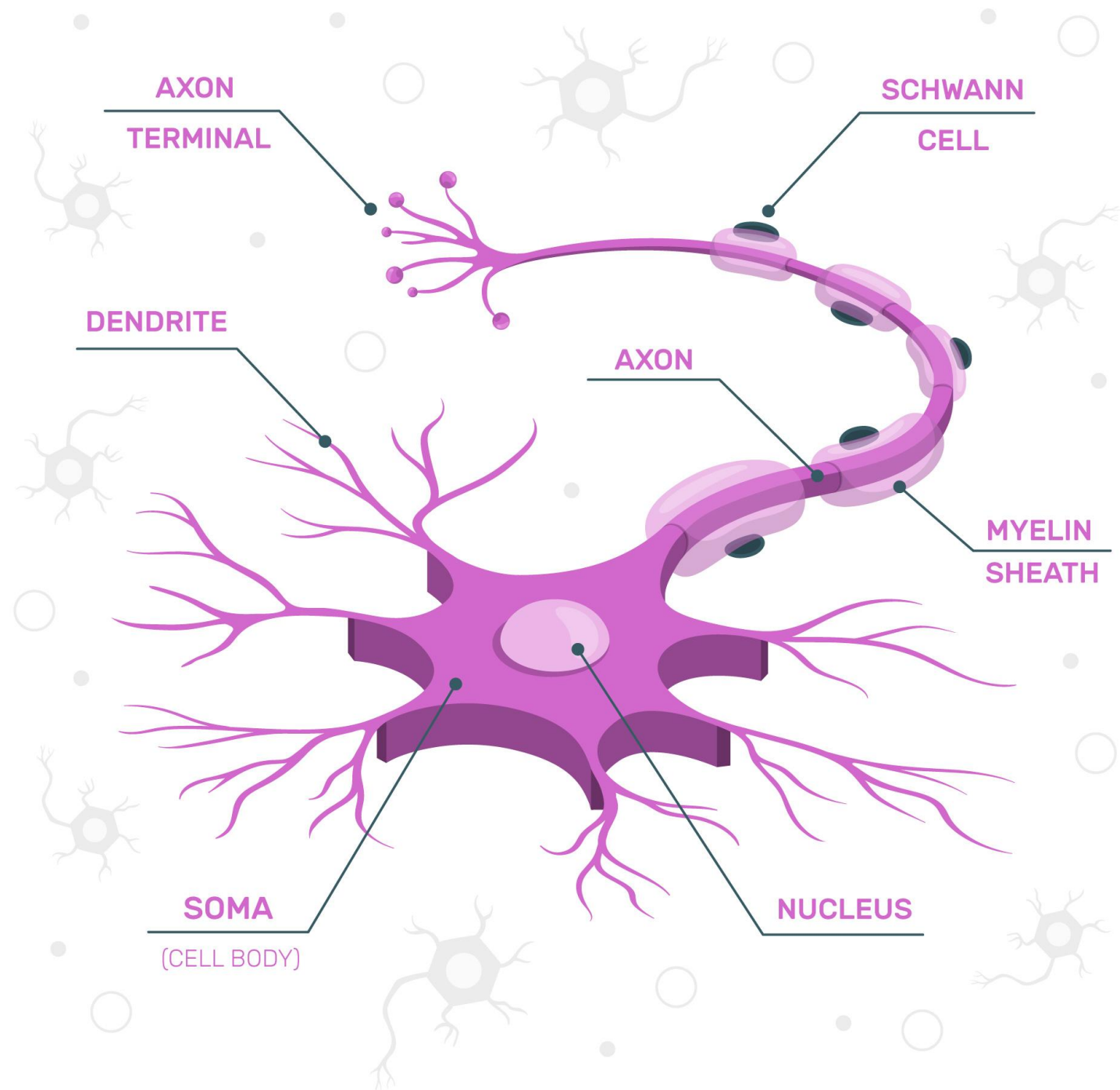


Que nos entregan:

- Poder identificar (clasificar) objetos
- Comparativas / Similitudes de objetos
- Mucha información de olores, sabores
- Patrones de comportamiento

¿Qué es una neurona biológica?

Este gráfico representa una **neurona biológica** y muestra las principales partes que la componen:



1. **Axon:** Es una estructura larga y delgada que transmite los impulsos eléctricos desde el soma (cuerpo celular) hacia otras neuronas o músculos. Es el camino que sigue la señal para ser enviada a otra célula.
2. **Axon Terminal:** Son las ramas finales del axón. Es donde ocurre la sinapsis, es decir, la comunicación entre la neurona y otras células. Aquí es donde se liberan neurotransmisores para transmitir señales a otras neuronas o músculos.
3. **Dendrite:** Las dendritas son las extensiones del soma que reciben señales de otras neuronas. Actúan como antenas que recogen la información de otras células.
4. **Soma (Cell Body):** También llamado cuerpo celular, contiene el núcleo de la célula y es responsable de mantener la célula funcionando. Aquí es donde se integra la información recibida de las dendritas.
5. **Nucleus:** El núcleo de la neurona contiene el material genético (ADN) de la célula y regula sus funciones.
6. **Myelin Sheath:** Es una capa aislante que recubre el axón. La mielina permite que los impulsos eléctricos viajen más rápidamente a lo largo del axón.
7. **Schwann Cell:** Son las células que producen la mielina en el sistema nervioso periférico. Están enrolladas alrededor del axón, formando la vaina de mielina.

Este gráfico es útil para mostrar cómo se estructura una neurona biológica y cómo se relacionan las diferentes partes para transmitir señales eléctricas dentro del sistema nervioso. En la comparación con una **neurona artificial**, los elementos como el **soma** podrían corresponder a la unidad de procesamiento central, y el **axón** sería equivalente al flujo de datos que viaja de una capa a otra en una red neuronal artificial.

En resumen

Hemos visto cómo las computadoras aprenden de los datos y nos ayudan a tomar decisiones comerciales. El Deep Learning es una parte más amplia de Machine Learning basado en redes neuronales artificiales que ayudan a resolver problemas complejos. Podemos reconocer imágenes, voces y expresiones faciales a través de mecanismos complejos, y muestra un éxito increíble en el entrenamiento de modelos tan intrincados.

Entendamos cómo funcionan fundamentalmente algunos de los mecanismos complejos:

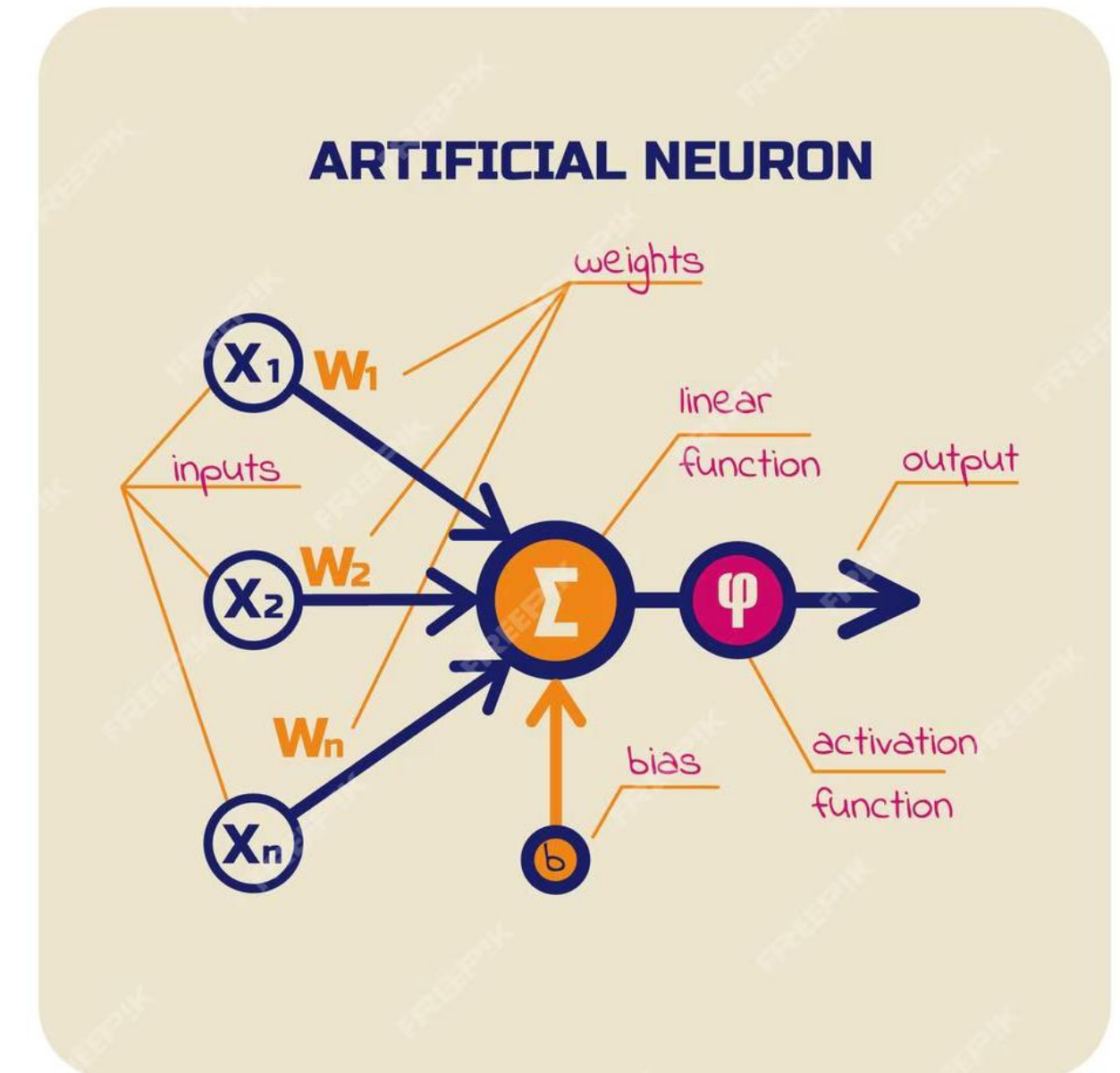
- Consideremos un ejemplo de un microprocesador, una unidad central de una computadora que procesa múltiples operaciones. Las operaciones lógicas fundamentales que funcionan detrás de un microprocesador son las puertas lógicas (puertas AND/OR). Se combinan múltiples puertas para realizar las tareas que hace un microprocesador.
- Otro ejemplo de una estructura compleja es la neurona. Está formada por las sinapsis, dendritas, cuerpos celulares y axones que forman el cerebro humano. Las dendritas captan señales de las neuronas vecinas, el soma suma todas las señales y el axón envía las señales a las células vecinas. Del mismo modo, tenemos miles de millones de neuronas en un cerebro humano, y estos miles de millones de neuronas están conectadas en una estructura compleja que permite que nuestro cerebro funcione. Así, inspirados por el sistema biológico, podemos tener una representación matemática de una neurona y entrenar a la máquina. **El componente fundamental de una red neuronal se llama perceptrón.**

¿Qué es una neurona artificial?

En una **neurona artificial**, las entradas se procesan mediante pesos y se suman para luego ser transformadas por una función de activación que determina la salida de la neurona.

1. **Inputs (X_1, X_2, X_n):** Las entradas (inputs) en una neurona artificial son equivalentes a las señales que reciben las dendritas. Son los datos que llegan desde otras neuronas o directamente del conjunto de datos.
2. **Weights (W_1, W_2, W_n):** Los pesos (weights) asignan una importancia a cada entrada. Determinan cuánto influye una entrada en el proceso de toma de decisiones de la neurona artificial.
3. **Función de agregación (Σ):** Todas las entradas ponderadas se combinan (sumatoria) para generar una señal agregada. Este paso es equivalente a la integración de señales en el cuerpo celular de una neurona biológica.
4. **Bias (b):** El sesgo (bias) es un valor adicional que se suma a la salida de la función de agregación. Ayuda a ajustar el modelo y permite que la neurona funcione incluso cuando todas las entradas son cero.
5. **Función de activación (ϕ):** Esta función transforma la señal agregada en una salida final. Puede ser una función no lineal como ReLU, sigmoide o tanh. Es lo que decide si la neurona se "activa" o no, similar a cómo las neuronas biológicas necesitan superar un umbral para enviar una señal.
6. **Output:** La salida final de la neurona artificial, que puede pasar a otras neuronas o capas en una red neuronal. En una red profunda, esta salida se convierte en la entrada de las siguientes capas.

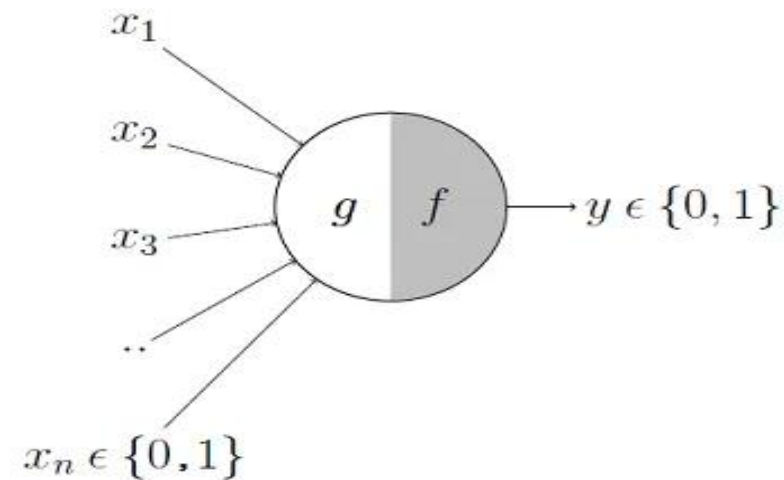
Esta comparación muestra cómo los conceptos biológicos de las neuronas han inspirado la creación de las **neuronas artificiales** en las redes neuronales, donde las entradas son datos, los pesos representan la importancia de esos datos, y las funciones de activación permiten que la red aprenda patrones complejos.



Perceptrones

El primer perceptrón fue escrito por el matemático y neurólogo McCulloch y Pitts en 1940. El perceptrón McCulloch-Pitts solo acepta entradas binarias y da la salida en binario, que utiliza una función escalonada.

Consideremos una neurona que tiene un montón de entradas, digamos x_1, x_2, x_3 , etc.



La neurona realiza dos tareas:

1. Primero hace la suma de las entradas $\sum x_i$
2. Luego aplica la función escalonada y verifica las siguientes condiciones.

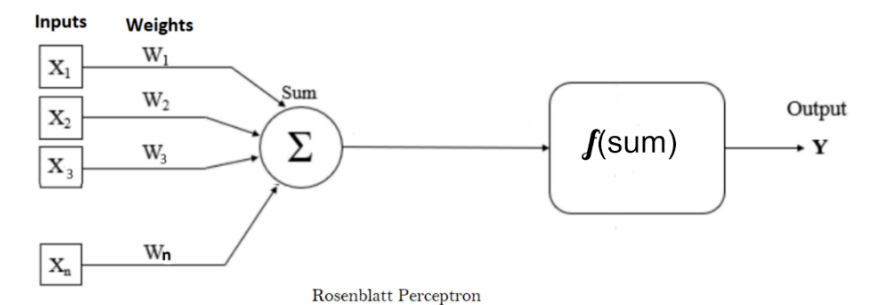
Si $\sum x_i \geq \theta$, da como salida 1

Si $\sum x_i < \theta$, da como salida 0

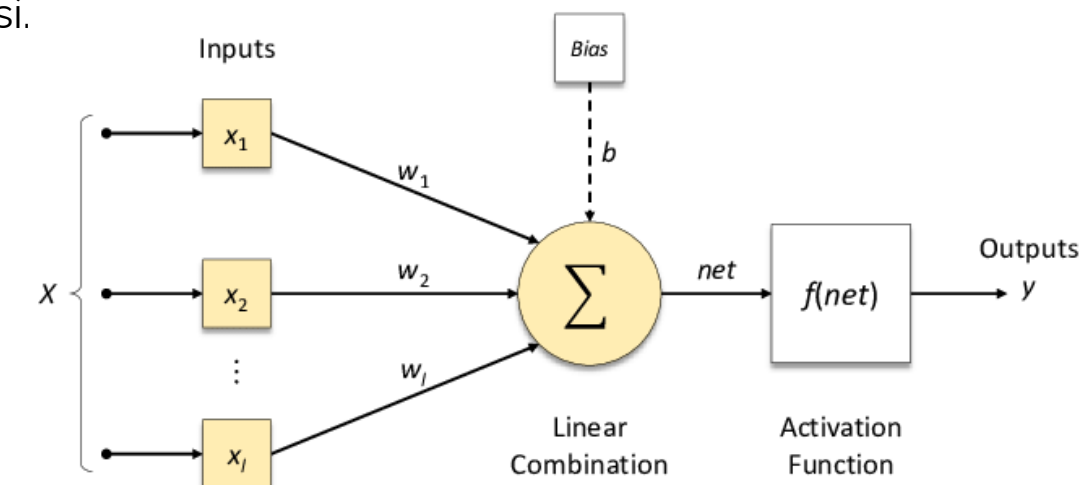
Este perceptrón podrá modelar diferentes puertas lógicas. Si $\theta = 1$, entonces funciona como una compuerta OR, donde cualquiera de las entradas debe ser 1, y si $\theta = 3$, será equivalente a una compuerta AND, donde las tres entradas deben ser iguales a 1. Algunas entradas también se pueden marcar como **entradas inhibitorias**. Si alguna de las entradas es inhibitoria, entonces inhiben inmediatamente la activación de la salida.

Este modelo de perceptrón fue la primera representación matemática de una neurona humana. El perceptrón **McCulloch-Pitts** tenía algunas limitaciones, como tomar solo entradas binarias y usar solo una función de activación de paso de umbral, que era ineficaz para aprender algo sobre los datos.

En 1958, Rosenblatt ideó otra versión del perceptrón que se usa hoy en día. Este perceptrón puede tomar cualquier número, no necesariamente binario. Todas las entradas tienen algunos pesos asociados con ellas y, en lugar de tomar la suma, toma la suma ponderada y aplica una función no lineal para generar la salida. El perceptrón se ilustra a continuación.



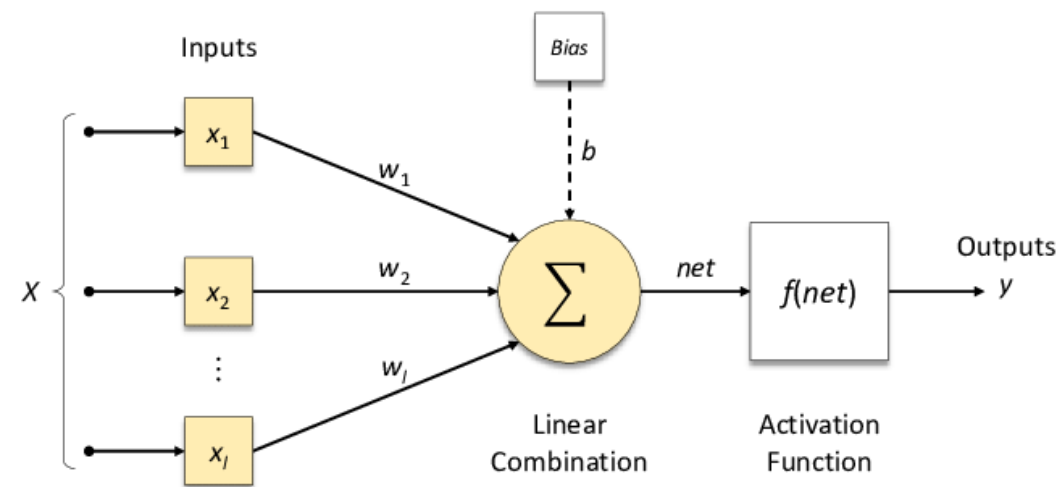
El propósito de agregar un término de sesgo al perceptrón es que theta sea parte de los pesos en lugar de parte de la función. El término de sesgo se agrega al perceptrón, que siempre es 1, y los pesos se ajustan manteniendo theta siempre en 0. Con un término de sesgo, la estructura del perceptrón se vería así.



Perceptron Multicapa

El aprendizaje profundo ayuda a resolver problemas complejos, y un perceptrón imita a la neurona humana para resolver problemas más simples. En lugar de tener un solo perceptrón podemos unir varios perceptrones y observamos cómo realizan una tarea.

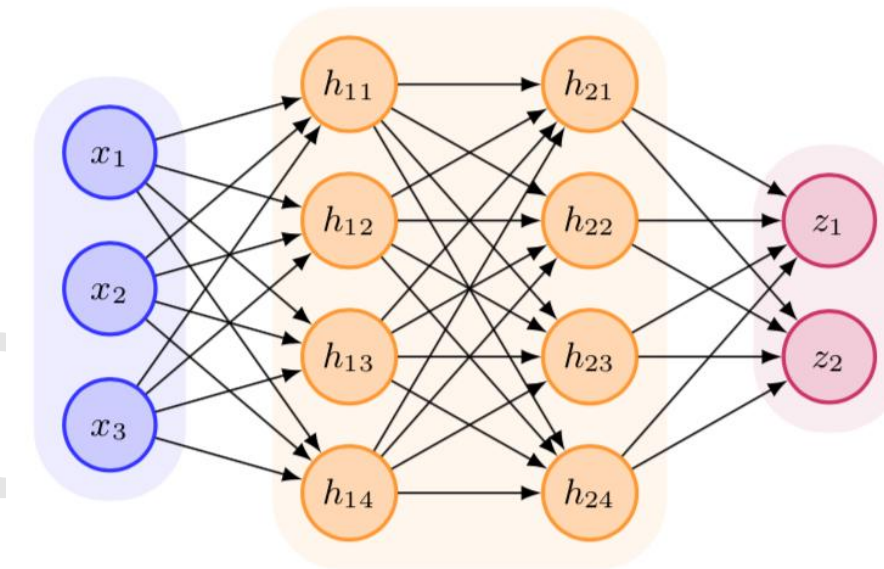
Considerando un solo perceptrón que toma algunas entradas, realiza la suma ponderada de las entradas y aplica la función sigmoidea. La función sigmoidea forma un gráfico en forma de S, donde x se acerca al infinito, la probabilidad se convierte en 1 y, a medida que x se acerca al infinito negativo, la probabilidad se convierte en 0.



La regresión logística funciona de la misma manera que un perceptrón único. Busca un modelo que toma la entrada, encuentra los coeficientes (como pesos del perceptrón) y predice el resultado. **Construir un modelo de regresión logística es similar a entrenar un perceptrón.**

Cuando los datos de entrenamiento X se pasan a un perceptrón, este devuelve valores predichos llamados $\hat{y}$, que deben ser lo más cercanos posible a los valores objetivo. Para lograr esto, la red neuronal comenzará con algunos pesos aleatorios y ajustará los pesos de manera iterativa al encontrar las derivadas con respecto a cada uno de los pesos para obtener las predicciones lo más cercanas posible a los valores reales.

Cuando conectamos varios perceptrones se forma una estructura **Deep neural network**. La primera capa se conoce como **Input Layer**, la capa final se conoce como **Output Layer** y las capas que se encuentran entre las capas de entrada y salida se conocen como **Hidden layer**. Cada capa podría tener una cantidad diferente de neuronas.

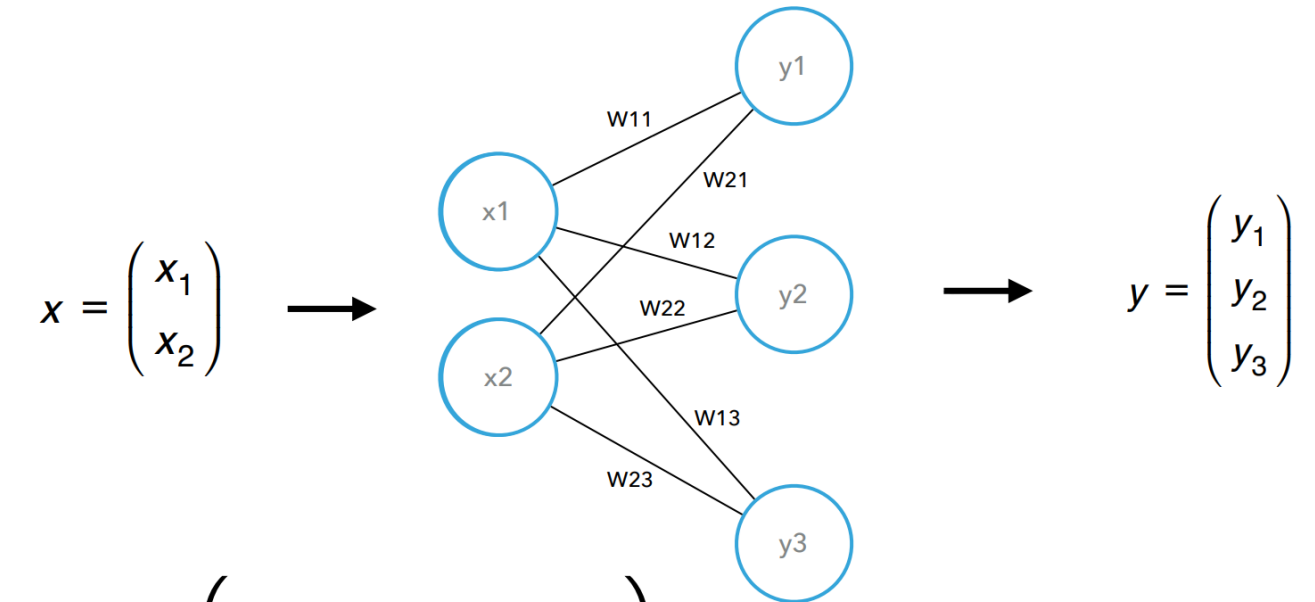


El término **bias** en la red neuronal es el mismo que la intersección en la ecuación de línea. El **bias** es el mismo que el valor **theta** o umbral en la neurona McCulloch Pitts. En lugar de tener este valor en la función, se agrega como parte de las entradas con un peso asociado a él.

Cada nodo (neurona) contiene un bias, que puede o no estar explícitamente mostrado en la arquitectura de la red neuronal. Una red neuronal tiene un conjunto de entradas y cada nodo tiene un peso. Calcula la suma ponderada de la entrada, aplica la función de activación, pasa la salida a la siguiente capa, y así sucesivamente.

Proceso de Aprendizaje de una Red Neuronal

- El proceso de multiplicar la entrada por los pesos de una red se llama Forward Propagation.



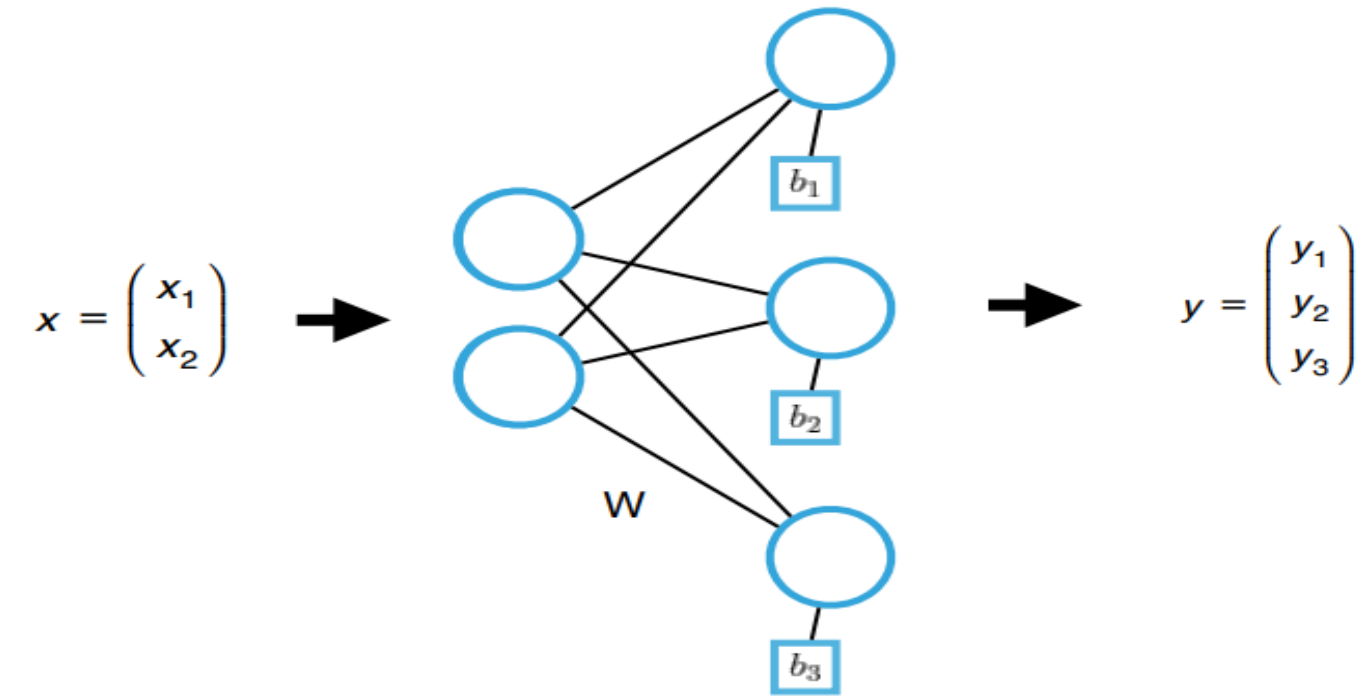
$$x \cdot W = (y_1, y_2, y_3)$$

- El proceso se basa en realizar la multiplicación de matrices entre capas

$$\begin{aligned} x \cdot W &= (x_1, x_2) \cdot \begin{pmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \end{pmatrix} \\ &= (x_1 w_{11} + x_2 w_{21}, x_1 w_{12} + x_2 w_{22}, x_1 w_{13} + x_2 w_{23}) \\ &= (y_1, y_2, y_3) \end{aligned}$$

Proceso de Aprendizaje de una Red Neuronal

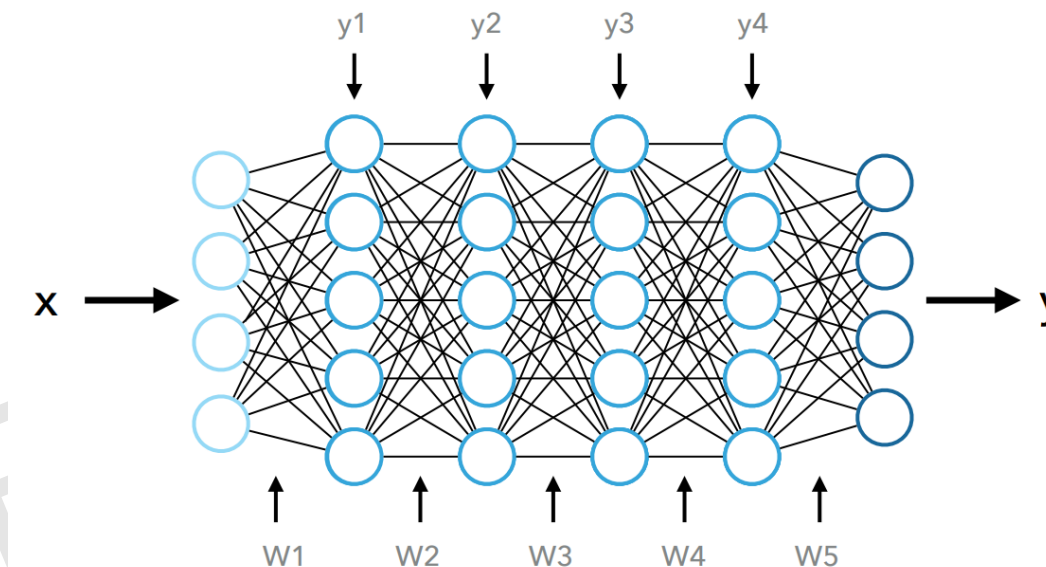
- Se suma un sesgo (bias) por neurona para ajustar el resultado.



$$\begin{aligned} x \cdot W + b &= (x_1, x_2) \cdot \begin{pmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \end{pmatrix} + (b_1, b_2, b_3) \\ &= (x_1x_{11} + x_2w_{21} + b_1, x_2x_{12} + x_2w_{22} + b_2, , x_3x_{13} + x_2w_{23} + b_3) \\ &= (y_1, y_2, y_3) \end{aligned}$$

Proceso de Aprendizaje de una Red Neuronal

- Cada neurona es un valor numérico resultado de la operación de multiplicar matrices de los pesos y los valores de la capa anterior.
- El valor de cada neurona consiste en una combinación lineal de los valores de la capa anterior ponderada por los pesos.
- Los pesos nos determinan cómo de fuerte es la conexión entre neuronas.



$$y_1 = x \cdot W_1 + b_1$$

$$y_2 = y_1 \cdot W_2 + b_2$$

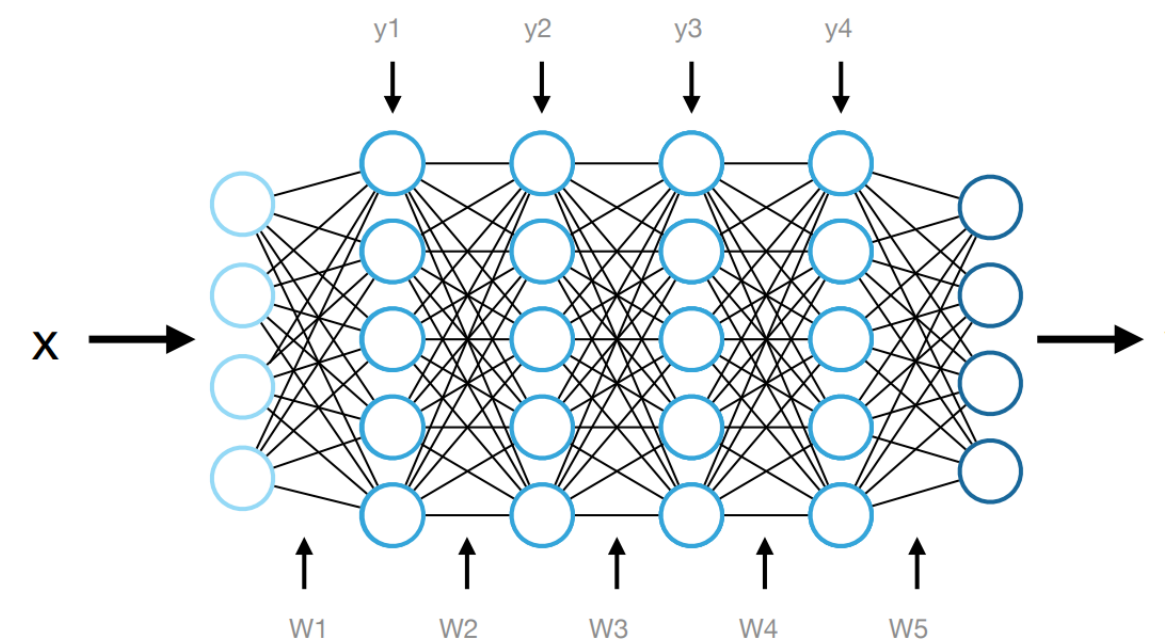
$$y_3 = y_2 \cdot W_3 + b_3$$

$$y_4 = y_3 \cdot W_4 + b_4$$

$$y = y_4 \cdot W_5 + b_5$$

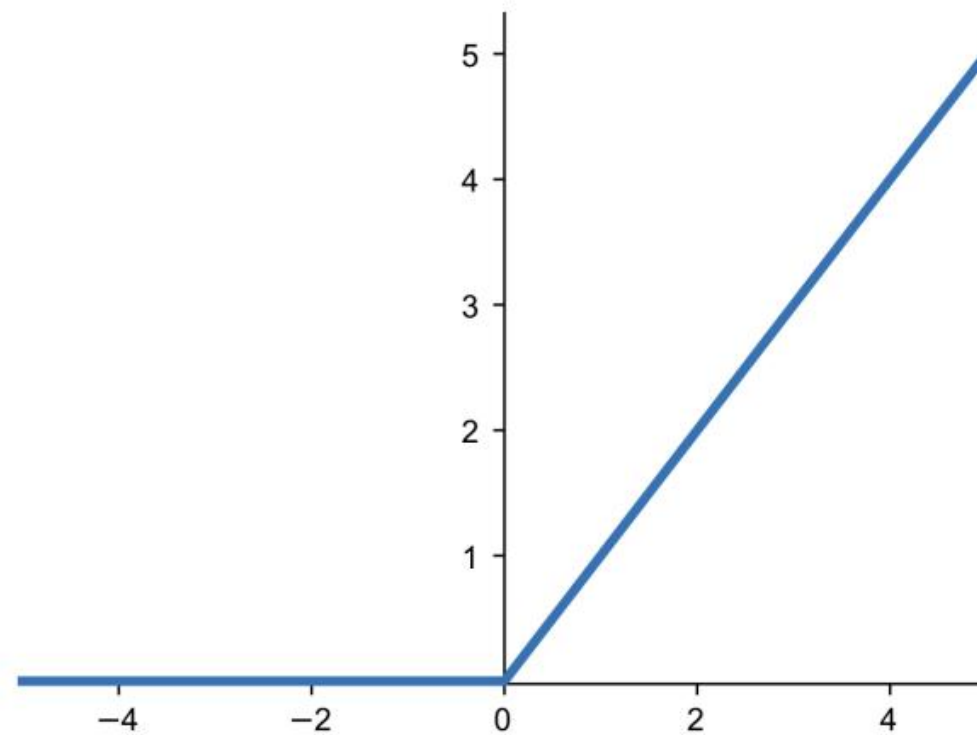
Proceso de Aprendizaje de una Red Neuronal

- Para abordar problemas complejos en redes neuronales, se ajusta la combinación lineal de los pesos y las entradas aplicando una **función de activación** en cada neurona.
- Las **funciones de activación** juegan un papel crucial en la red neuronal, ya que son las responsables de decidir si una neurona debe activarse o no en función de la salida procesada. Si la neurona se activa, transmite la señal a las siguientes capas de la red. Sin esta activación, las capas profundas no podrían capturar relaciones no lineales entre las variables de entrada y salida, lo que es esencial para resolver tareas complejas, como el reconocimiento de imágenes o el procesamiento de lenguaje natural.
- Las funciones de activación más comunes son: ReLU, Sigmoid y Tanh.



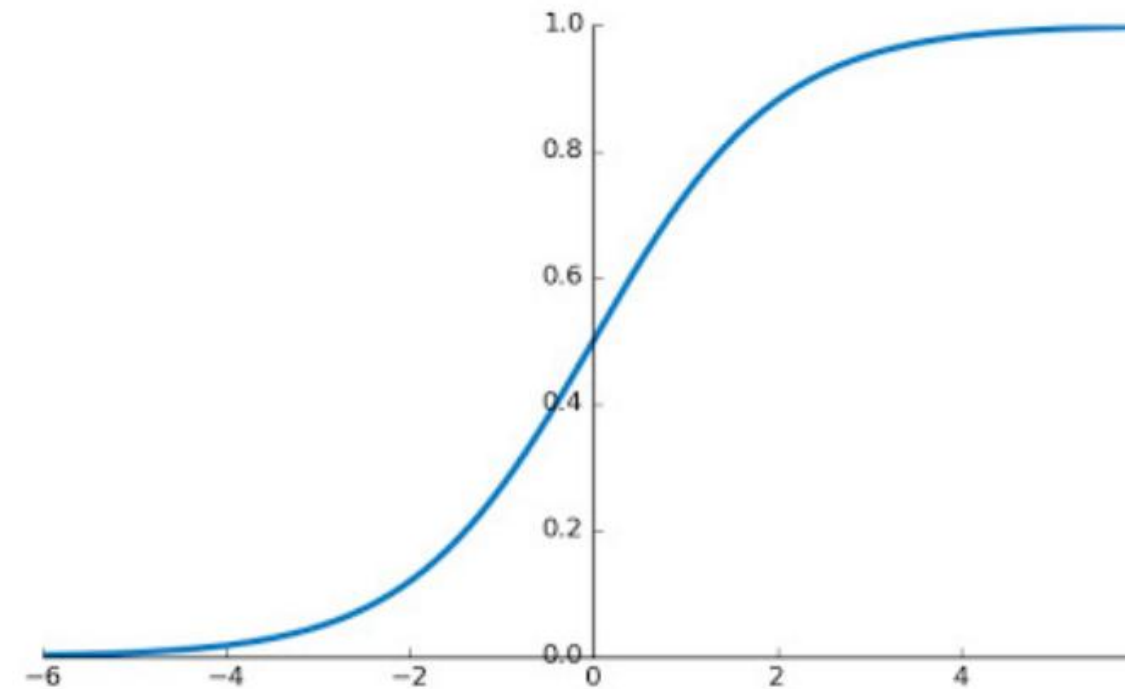
$$\begin{aligned}y_1 &= \sigma(x \cdot W_1 + b_1) \\y_2 &= \sigma(y_1 \cdot W_2 + b_2) \\y_3 &= \sigma(y_2 \cdot W_3 + b_3) \\y_4 &= \sigma(y_3 \cdot W_4 + b_4) \\y &= \sigma(y_4 \cdot W_5 + b_5)\end{aligned}$$

Proceso de Aprendizaje de una Red Neuronal



Función ReLU

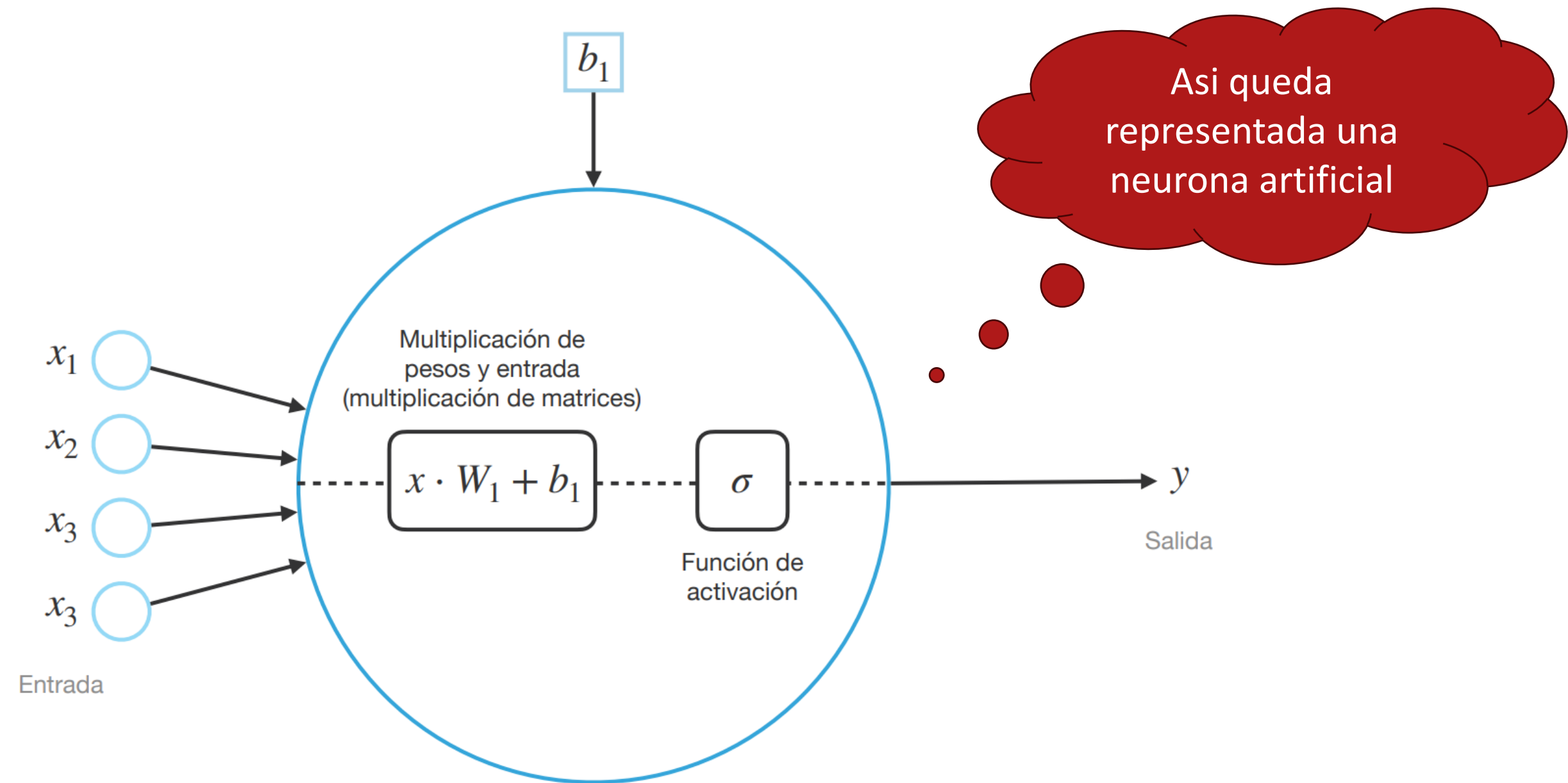
$$f(x) = \max(0, x)$$



Función Sigmoid

$$f(x) = \frac{1}{1 + e^{-x}}$$

Proceso de Aprendizaje de una Red Neuronal



Funciones de Activación

Una **función de activación** es un componente clave en una red neuronal, que se utiliza para transformar la salida de una neurona en un valor que luego se pasa a la siguiente capa de la red. En esencia, la función de activación decide si una neurona debe activarse o no, es decir, si su salida debe ser transmitida a las siguientes capas.

¿Por qué es importante una función de activación?

Sin una función de activación, la salida de una neurona sería simplemente una combinación lineal de las entradas y los pesos, lo que convertiría la red neuronal en un simple modelo lineal. Las funciones de activación introducen **no linealidad**, lo que permite que la red aprenda y represente patrones complejos en los datos, como relaciones no lineales.

Propósito de la función de activación

- 1. Introducir no linealidad:** Permiten a la red neuronal aprender funciones no lineales y representar relaciones complejas en los datos. Sin estas funciones, la red sería incapaz de aprender patrones no lineales.
- 2. Controlar la activación de neuronas:** La función de activación define qué neuronas deben activarse (es decir, pasar la información a la siguiente capa) y cuáles deben quedarse "apagadas". Esto ayuda a la red a identificar las características más importantes de los datos.

Tipos de funciones de activación

1. **Step Function (Función Escalón)**
2. **Sigmoid Activation**
3. **Tanh (Tangente hiperbólica)**
4. **ReLU (Rectified Linear Unit).**
5. **Linear Activation Function (Función de Activación Lineal)**
6. **Leaky ReLU (Rectified Linear Unit con fuga)**
7. **Softmax**

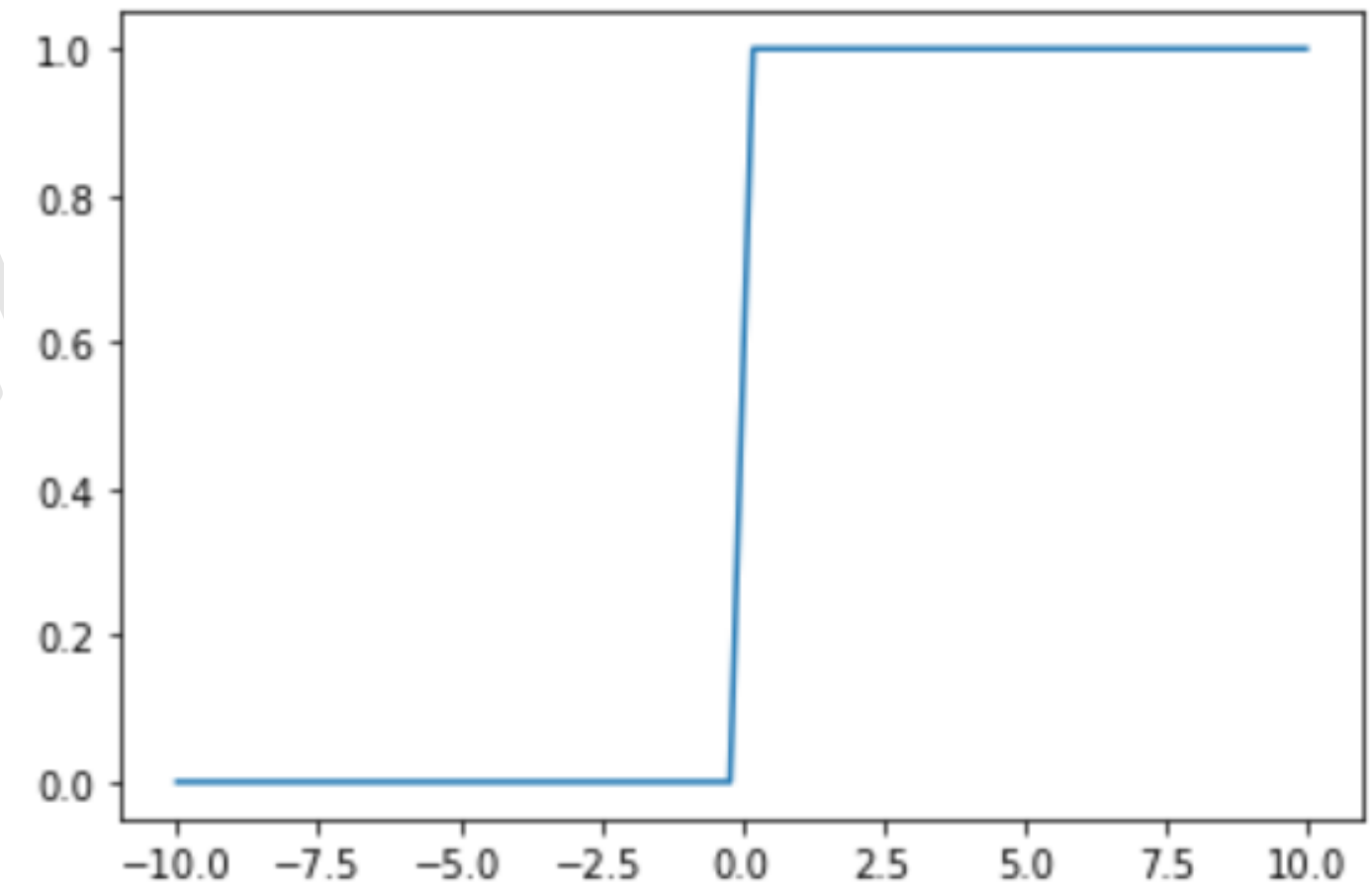
Step function

La **Step Function** o función escalón es una de las funciones de activación más simples. Transforma la salida de una neurona en dos posibles valores, generalmente 0 o 1, dependiendo de si la entrada es menor o mayor que un umbral.

Una función escalonada binaria es una función de activación basada en un umbral, lo que significa que se activa cuando se alcanza un determinado umbral y se desactiva cuando cae por debajo de ese punto. Debido a que hay un salto brusco en la función, la derivada en $x=0$ dará un número grande, pero ese número grande no es útil para el aprendizaje y no se puede utilizar para la clasificación de múltiples clases. Las funciones suaves son buenas para el aprendizaje cuando hay pendientes definidas y esta función escalonada es una función brusca.

- ❖ **Ventaja:** Es fácil de implementar y puede ser utilizada en tareas de clasificación binaria. Funciona bien cuando se requiere una activación estricta (activación o no activación).
- ❖ **Desventaja:** No es adecuada para modelos de *deep learning*, ya que es una función no diferenciable, lo que impide que se calcule el gradiente y dificulta el proceso de entrenamiento mediante retro propagación.
- ❖ **Uso actual:** Aunque históricamente se usaba en las primeras redes neuronales (como los perceptrones), ya no es común en redes modernas debido a la falta de diferenciability. Modelos básicos de clasificación binaria (obsoleta en deep learning)
- ❖ **Fórmula**

$$f(x) = \begin{cases} 0 & \text{si } x < 0 \\ 1 & \text{si } x \geq 0 \end{cases}$$



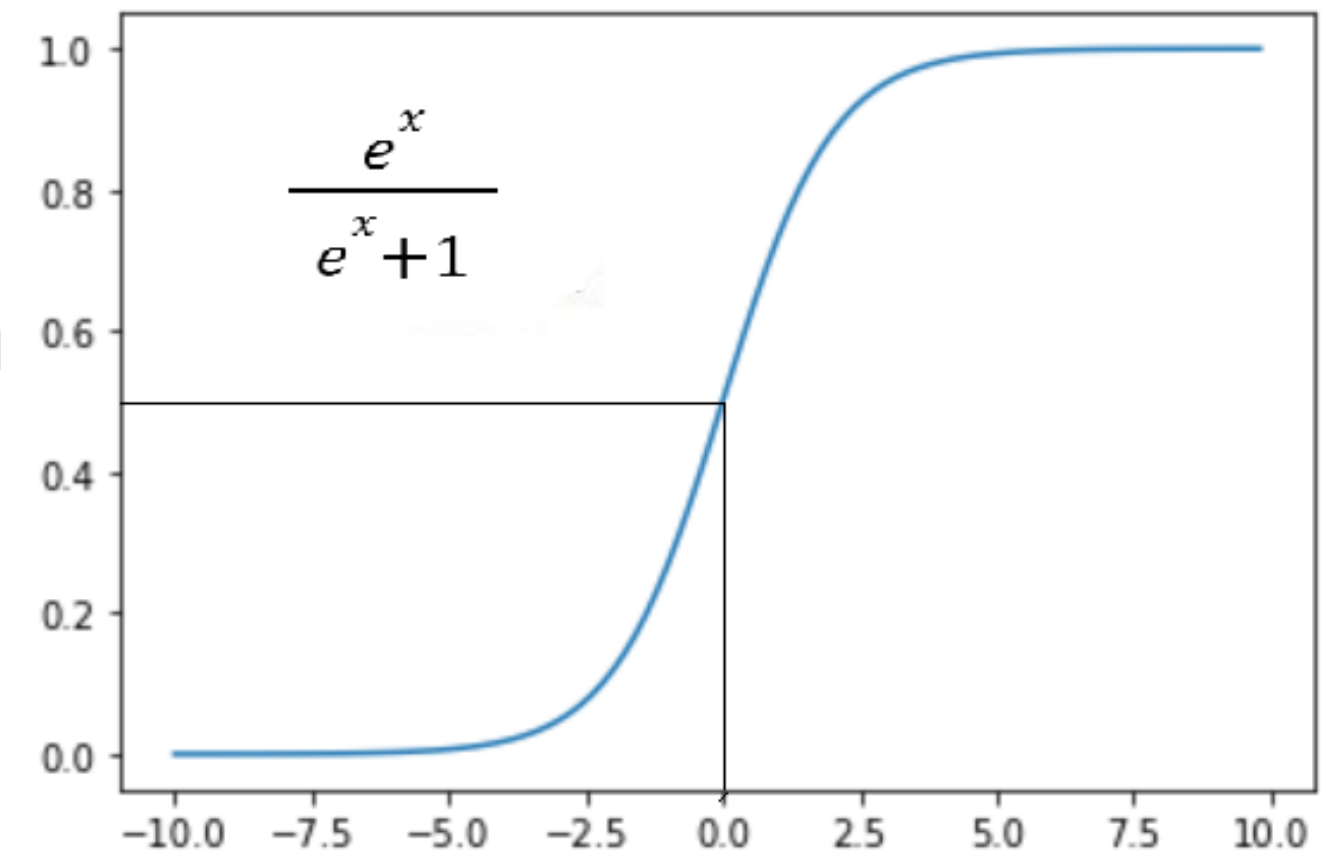
Sigmoid Activation Function

La **función sigmoide** es una función de activación no lineal que convierte la entrada en un valor entre 0 y 1. Es útil en problemas de clasificación binaria, donde queremos que la salida sea una probabilidad (es decir, un valor entre 0 y 1). La función sigmoide es especialmente útil en la capa de salida para modelos de clasificación binaria.

Utiliza un enfoque probabilístico. Es una función que se representa gráficamente en forma de "S". Debido a que los valores de la función sigmoidea varían entre 0 y 1, se puede predecir fácilmente que el resultado será 1 si el valor es mayor que 0,5 y 0 en caso contrario. En la capa de salida de una clasificación binaria, normalmente se utilizan valores de la función sigmoidea, con un resultado que puede ser 0 o 1.

- ❖ **Ventaja:** Produce una salida interpretada como una probabilidad, lo que es útil para tareas de clasificación. Es suave y diferenciable, lo que facilita la propagación hacia atrás (backpropagation).
- ❖ **Desventaja:** La **saturación** en los extremos (cuando x es muy grande o muy pequeño) provoca que el gradiente sea muy pequeño, lo que puede ralentizar el entrenamiento debido al **gradiente desvanecido**. La salida no está centrada en cero, lo que puede causar problemas de convergencia..
- ❖ **Uso actual:** Última capa en modelos de clasificación binaria.
- ❖ **Fórmula:** Donde x es la entrada a la neurona.

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



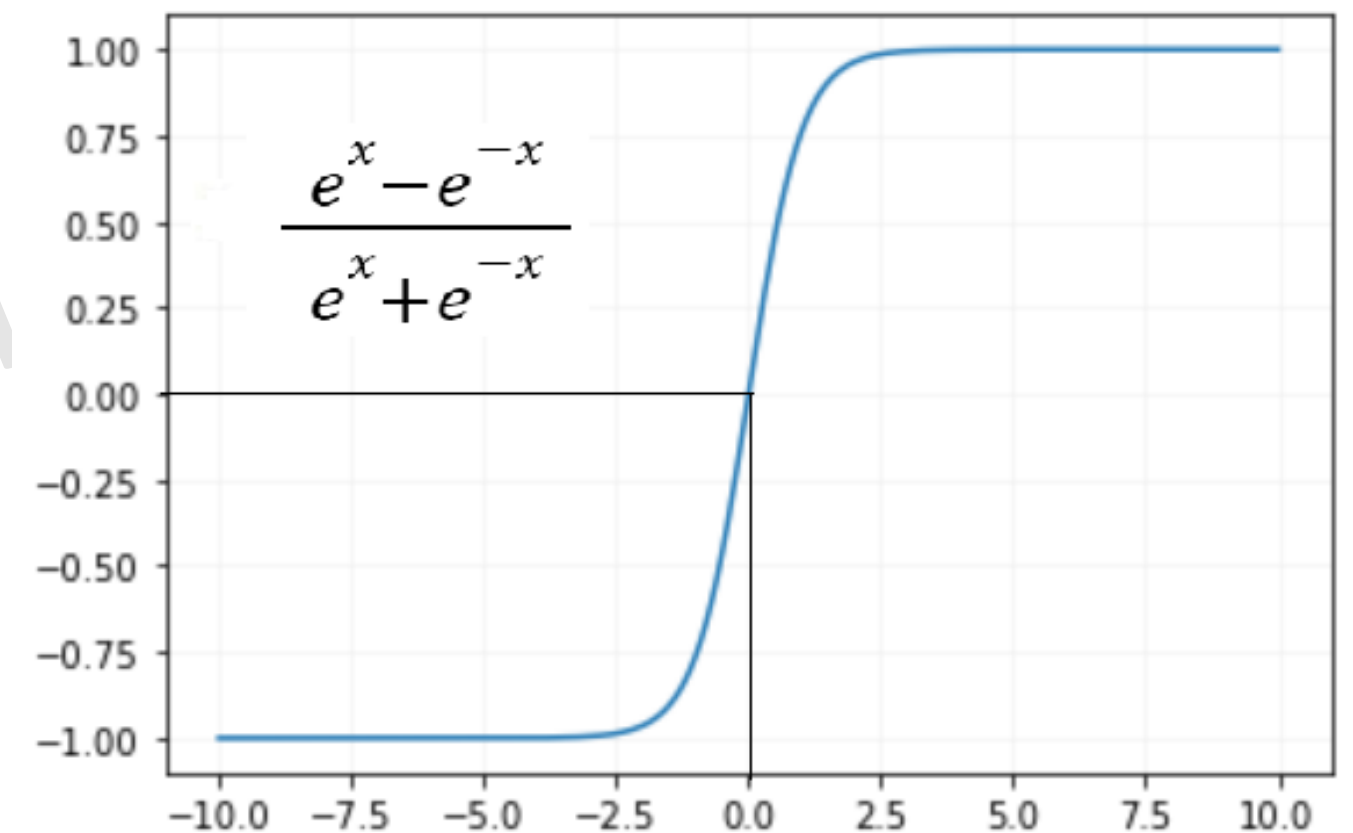
Tanh Function

La función **tanh** es otra función de activación no lineal, que convierte la entrada en un valor entre -1 y 1. Al igual que la sigmoide, es suave y diferenciable, pero tiene la ventaja de estar centrada en cero, lo que facilita la convergencia en las primeras capas.

La función de activación Tanh es una versión comprimida y escalada de la función de activación sigmoidea y también se denomina tangente hiperbólica. Significativamente superior a la sigmoide porque permite salidas negativas y tiene un rango de salida de -1 a 1.

- ❖ **Ventaja:** Está centrada en cero, lo que facilita la actualización de los pesos en redes neuronales. Puede ser útil en redes recurrentes o en problemas donde la simetría alrededor del origen es importante.
- ❖ **Desventaja:** Al igual que la función sigmoide, sufre del problema de **gradiente desvanecido**, lo que puede ralentizar el aprendizaje en redes profundas. Su computación es más costosa que ReLU debido a su naturaleza exponencial.
- ❖ **Uso actual:** Capas ocultas en redes recurrentes y modelos simétricos
- ❖ **Fórmula:**

$$\tanh(x) = \frac{2}{1 + e^{-2x}} - 1$$



ReLU Function

La **función de activación ReLU** es actualmente una de las más utilizadas en Deep learning en las hidden layers y su rango va de 0 a infinito. La razón principal de su popularidad es su simplicidad y eficiencia. ReLU pasa las entradas positivas sin cambios y asigna cero a las entradas negativas.

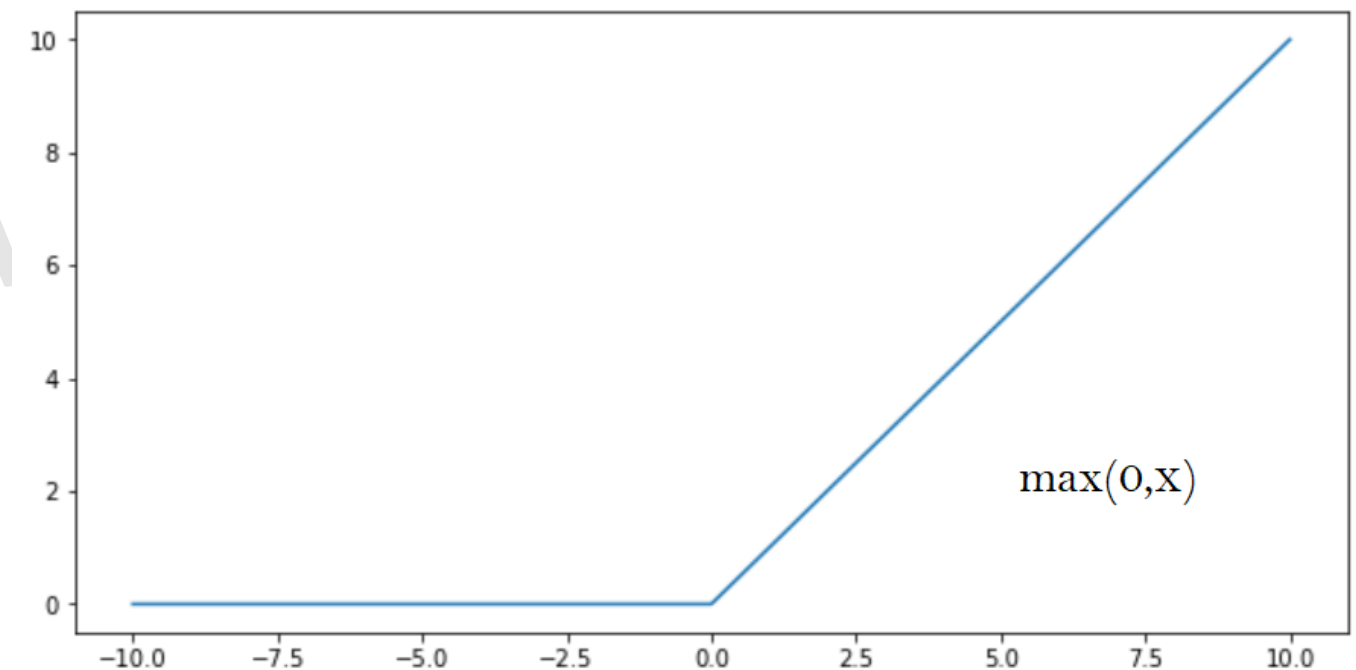
si $X \geq 0$, X

si $X < 0$, 0

Si x es positivo, genera x , y si no, genera 0

- ❖ **Ventaja:** Es eficiente en términos computacionales porque no requiere operaciones exponenciales. Resuelve el problema del gradiente desvanecido al mantener un gradiente constante para entradas positivas. Aumenta la **esparsidad** en la activación de neuronas, lo que ayuda a mejorar la eficiencia de la red.
- ❖ **Desventaja:** Puede llevar a un problema conocido como **neurona muerta** (dead neurons), donde ciertas neuronas nunca se activan (es decir, tienen siempre una salida de 0), especialmente si reciben valores negativos de manera continua. Es sensible a la inicialización de los pesos.
- ❖ **Uso actual:** Capas ocultas en redes profundas
- ❖ **Fórmula:** Donde x es la entrada a la neurona.

$$f(x) = \max(0, x)$$



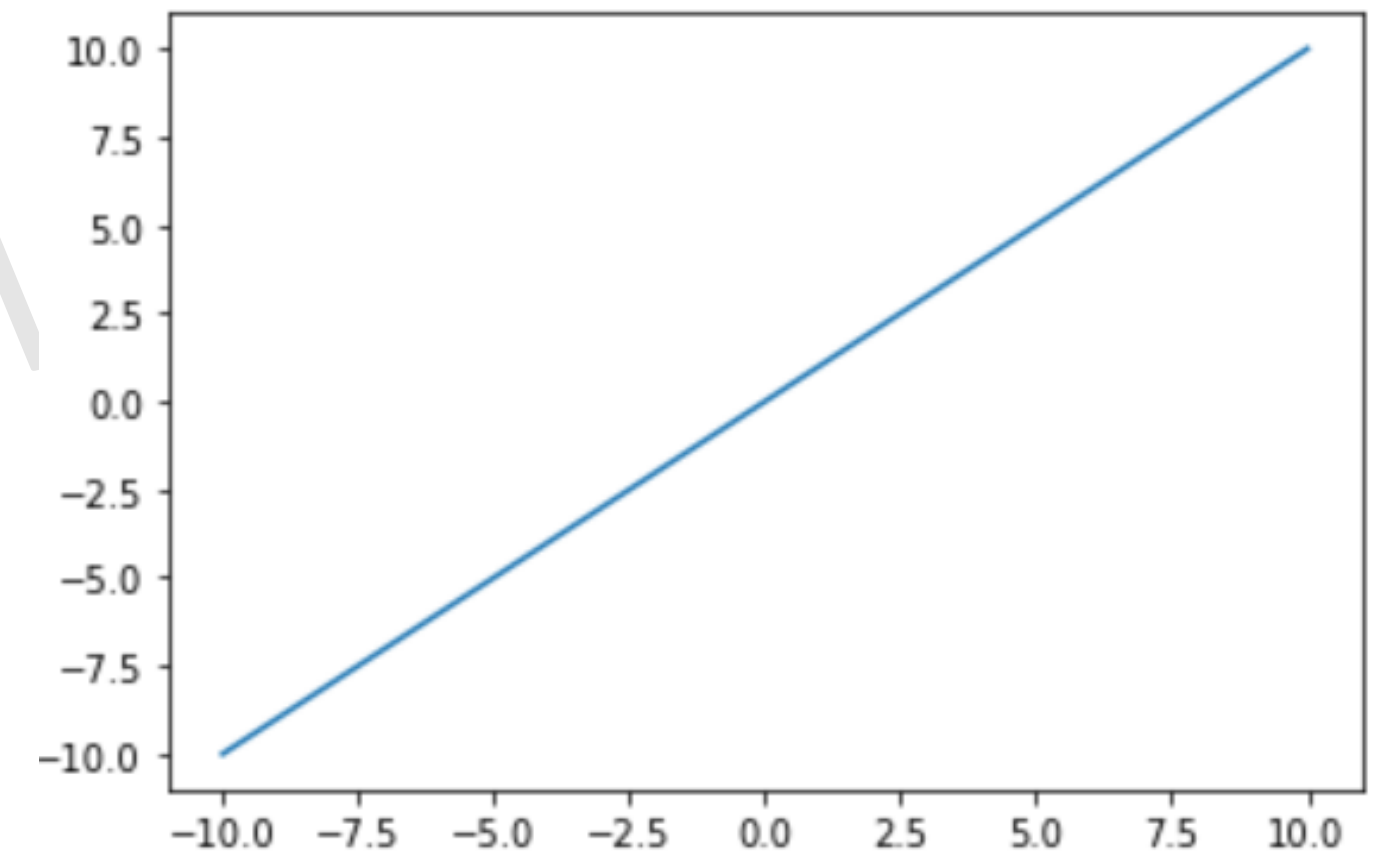
Linear activation Function

La **función de activación lineal** simplemente devuelve el valor de la entrada sin ningún cambio. Es útil en ciertos contextos, pero tiene limitaciones significativas cuando se usa en redes profundas. La función de activación lineal es aquella en la que la activación es proporcional a la entrada. Esta función no hace nada con la suma ponderada de la entrada y devuelve el valor que se le dio.

La función de activación de la última capa es simplemente una función lineal de la entrada de la primera capa, independientemente de cuántas capas haya, suponiendo que todas sean lineales. La función de activación lineal tiene un rango de $-\infty$ a $+\infty$. La última capa de la red neuronal funcionará como una función lineal de la primera capa.

- ❖ **Ventaja:** Sencilla y directa. Puede ser útil en redes donde se necesita mantener la escala de la salida, como en modelos de regresión.
- ❖ **Desventaja:** No introduce no linealidad, por lo que si todas las capas de una red usan una función de activación lineal, la red entera es equivalente a una sola capa lineal. Esto limita severamente la capacidad de la red para aprender patrones complejos y no lineales en los datos.
- ❖ **Uso actual:** Se utiliza principalmente en la capa de salida de modelos de regresión, donde se requiere una salida continua no limitada.
- ❖ **Fórmula:** Donde x es la entrada a la neurona.

$$f(x) = x$$



Leaky ReLU Function

La **Leaky ReLU** es una variante de la función **ReLU** que intenta resolver el problema de las "neuronas muertas" que se presenta en **ReLU** estándar, donde las neuronas que reciben entradas negativas permanecen inactivas para siempre (saliendo un 0). En **Leaky ReLU**, las entradas negativas no se eliminan por completo, sino que se les asigna un pequeño valor negativo.

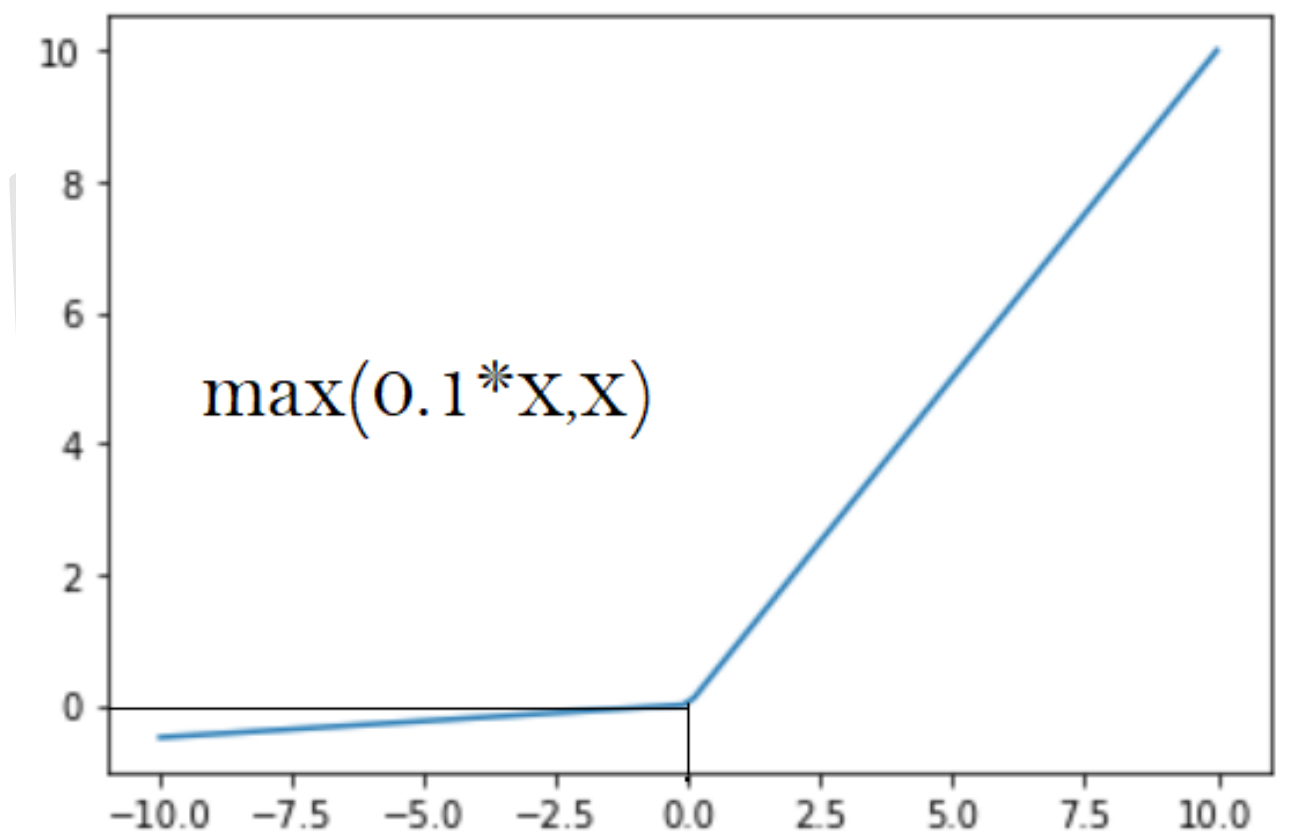
La función Leaky ReLU se define para abordar este problema. En lugar de definir la función de activación ReLU como 0 para valores negativos de entradas (x), la definimos como un componente lineal extremadamente pequeño de x . Aquí está la fórmula para esta función de activación.

$$f(x) = \max(0.01 * x, x)$$

Si la función anterior recibe cualquier entrada positiva, devuelve x ; de lo contrario, devuelve un valor pequeño igual a 0.001 veces x . Como resultado, produce una salida para cualquier valor negativo. Al realizar este pequeño cambio, el gradiente en el lado izquierdo del gráfico anterior deja de ser cero. Como resultado, ya no encontraría neuronas muertas en esa área.

- ❖ **Ventaja:** Soluciona el problema de las neuronas muertas al permitir un pequeño gradiente cuando la entrada es negativa. Esto ayuda a que las neuronas sigan aprendiendo incluso si reciben entradas negativas.
- ❖ **Desventaja:** Aunque mejora sobre **ReLU**, en algunos casos no se justifica su uso si no se observa un problema grave de neuronas moribundas.
- ❖ **Uso actual:** Redes profundas, capas ocultas
- ❖ **Fórmula:** Donde α es un valor pequeño, típicamente $\alpha=0.01$.

$$f(x) = \begin{cases} x & \text{si } x > 0 \\ \alpha x & \text{si } x \leq 0 \end{cases}$$



Softmax Activation Function

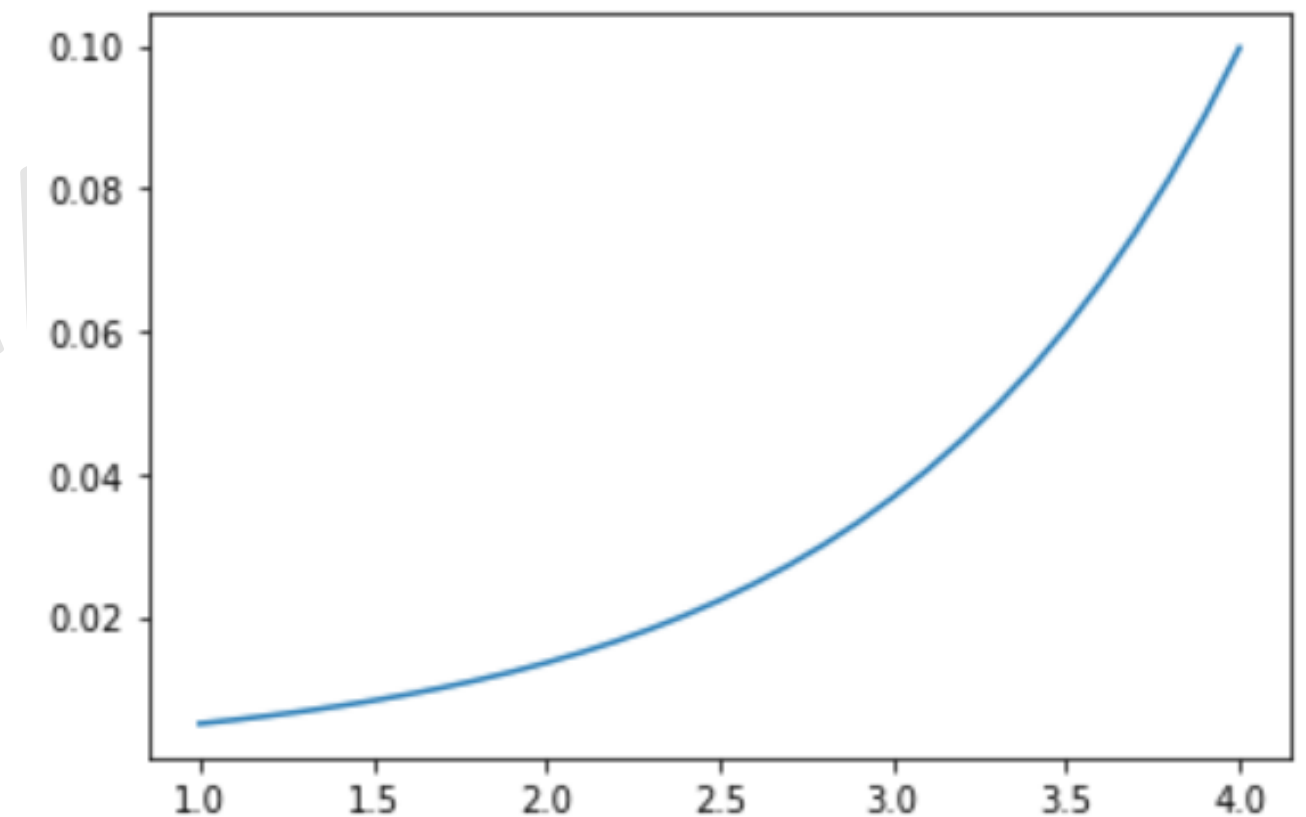
La función **Softmax** se utiliza principalmente en la capa de salida de los modelos de clasificación multiclase. Transforma un vector de valores numéricos (es decir, las salidas de las neuronas) en un vector de probabilidades, donde la suma de todas las probabilidades es igual a 1. Es especialmente útil para problemas de clasificación multiclase, donde cada clase se representa como una probabilidad de que un punto de datos pertenezca a cada clase individual. El rango de softmax es $-\infty$ a ∞ .

La función softmax se describe a menudo como una combinación de múltiples sigmoides. La función de activación sigmoidea devuelve valores entre 0 y 1, que son las probabilidades de que cada uno de los puntos de datos pertenezca a una clase particular. Por lo tanto, la función sigmoidea se utiliza ampliamente para problemas de clasificación binaria.

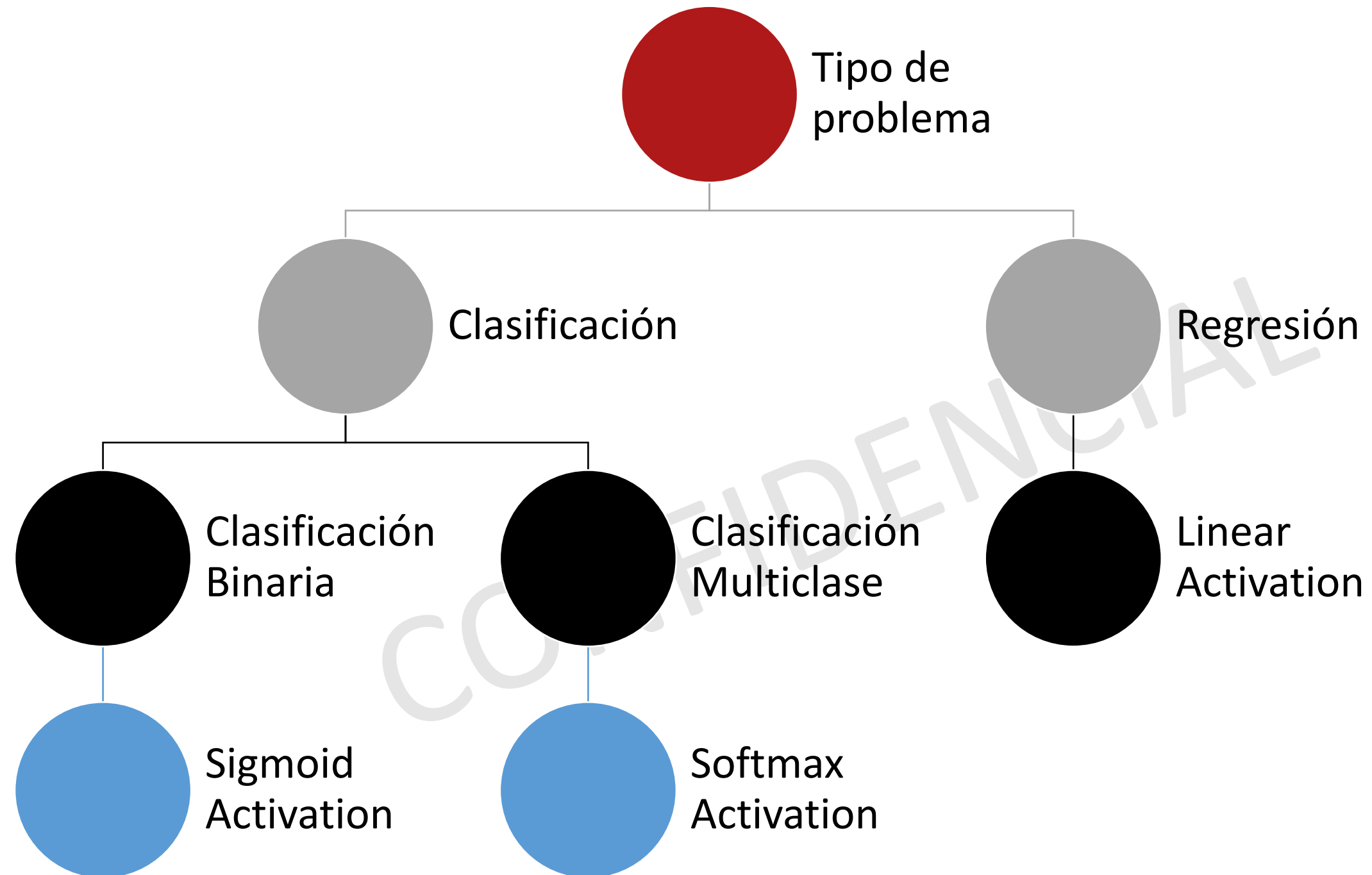
La función normaliza las salidas para cada clase entre 0 y 1 y suma la suma normalizada para cada clase. La salida para una clase particular es su salida normalizada dividida por la suma normalizada total.

- ❖ **Ventaja:** Convierte las salidas en probabilidades interpretables, lo que es crucial para problemas de clasificación. Es diferenciable y, por lo tanto, es compatible con técnicas de retropropagación.
- ❖ **Desventaja:** Puede sufrir de **saturación** para entradas extremadamente grandes o pequeñas, lo que podría llevar a valores de gradiente muy pequeños para algunas clases. Puede ser computacionalmente costosa en problemas con muchas clases.
- ❖ **Uso actual:** Última capa en modelos de clasificación multiclase
- ❖ **Fórmula:** Donde x_i representa la salida de la neurona i .

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}$$



Elegir la función de activación correcta en la capa de salida



Loss Function o funcion de pérdida

Es una métrica que mide qué tan bien o mal está funcionando un modelo de aprendizaje automático al comparar sus predicciones con los valores reales o esperados (también conocidos como **Ground Truth**). La función de pérdida calcula la **diferencia** entre los valores predichos por el modelo y los valores reales, y devuelve un número que indica el **error** del modelo. El objetivo del proceso de entrenamiento es minimizar este error para mejorar la precisión de las predicciones.

Es uno de los componentes más importantes de las redes neuronales ya que mide la proximidad de un vector (valores reales) a otro vector (valores predichos, $\hat{y}$).

¿Por qué es importante una loss function?

La función de pérdida es crucial para entrenar un modelo, ya que proporciona una manera de evaluar qué tan cerca están las predicciones del modelo respecto a los valores reales. A través de este valor de error, el modelo puede ajustar sus parámetros (como los pesos en una red neuronal) para reducir el error y hacer predicciones más precisas.

Mean Absolute Error

Mean Squared Error (MSE) (Error Cuadrático Medio)

Mean Squared Logarithmic Error (MSLE)

Binary Cross-Entropy (Entropía Cruzada Binaria)

Hinge Loss

Squared Hinge Loss

Categorical Cross-Entropy

Sparse Categorical Cross Entropy

Kullback-Leibler Divergence (KL Divergence)

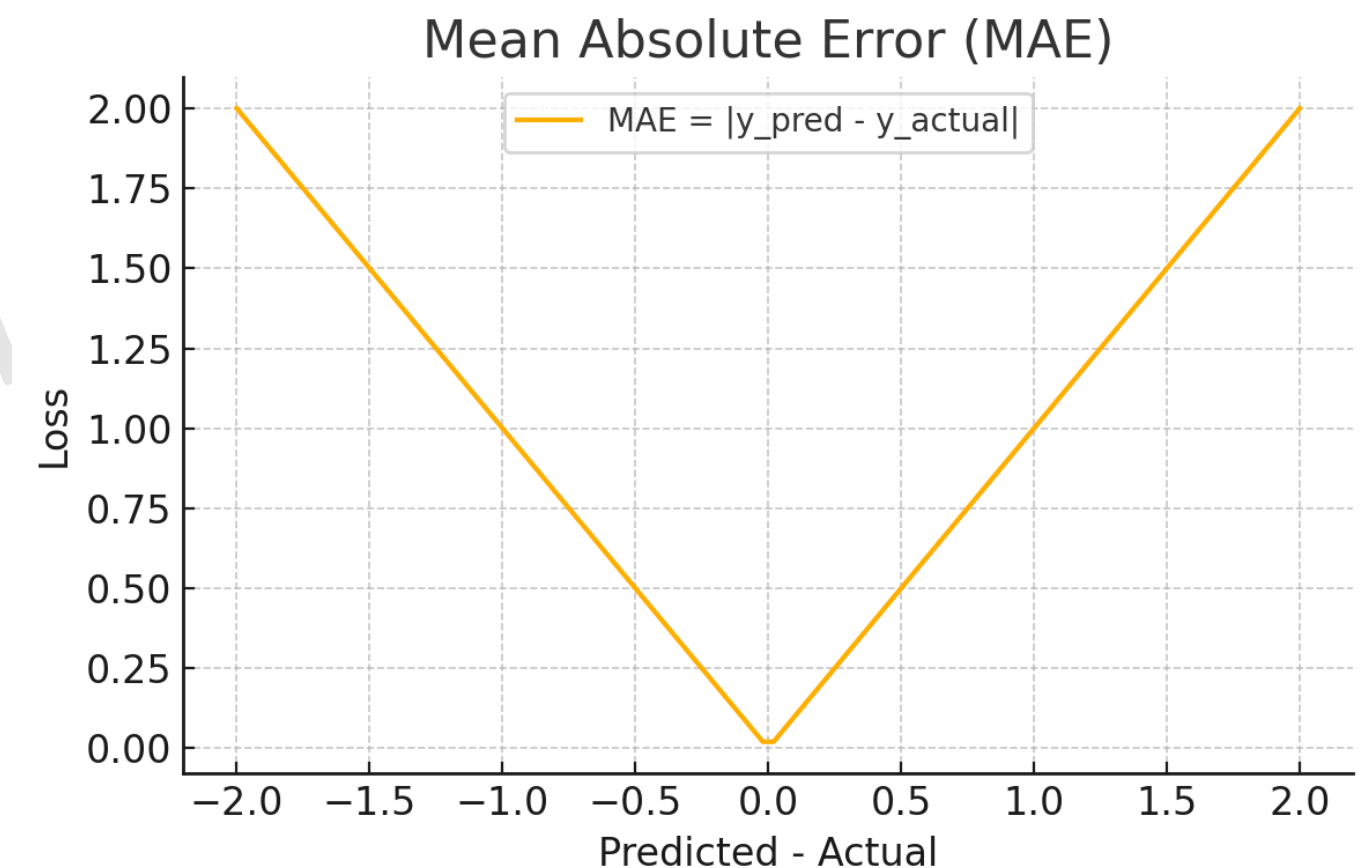
La **Mean Absolute Error (MAE)** es una función de pérdida que mide el error promedio en términos absolutos. Se utiliza en problemas de **regresión** y es más robusta frente a valores atípicos en comparación con el **Mean Squared Error (MSE)**, ya que no da un peso cuadrado a los errores grandes.

La capacidad de diferenciar la función es muy importante para optimizar la función de pérdida, y el problema con los valores absolutos es que la función no está definida en $x = 0$ considerando la gráfica de la función absoluta.

Esta función absoluta no es eficiente para el entrenamiento porque, en $x=0$, no está definida. Como será difícil diferenciar la función absoluta, **se utiliza un cuadrado de la función de pérdida en lugar de la función absoluta, que también se denomina pérdida l2 o error cuadrático medio**.

- ❖ **Ventaja:** Penaliza todos los errores de manera lineal, lo que la hace menos sensible a valores atípicos. Fácil de interpretar, ya que mide el error absoluto promedio.
- ❖ **Desventaja:** Puede ser menos preciso en modelos que requieren sensibilidad a grandes errores.
- ❖ **Uso actual:** Regresión cuando no se desea penalizar excesivamente los grandes errores.
- ❖ **Fórmula:**

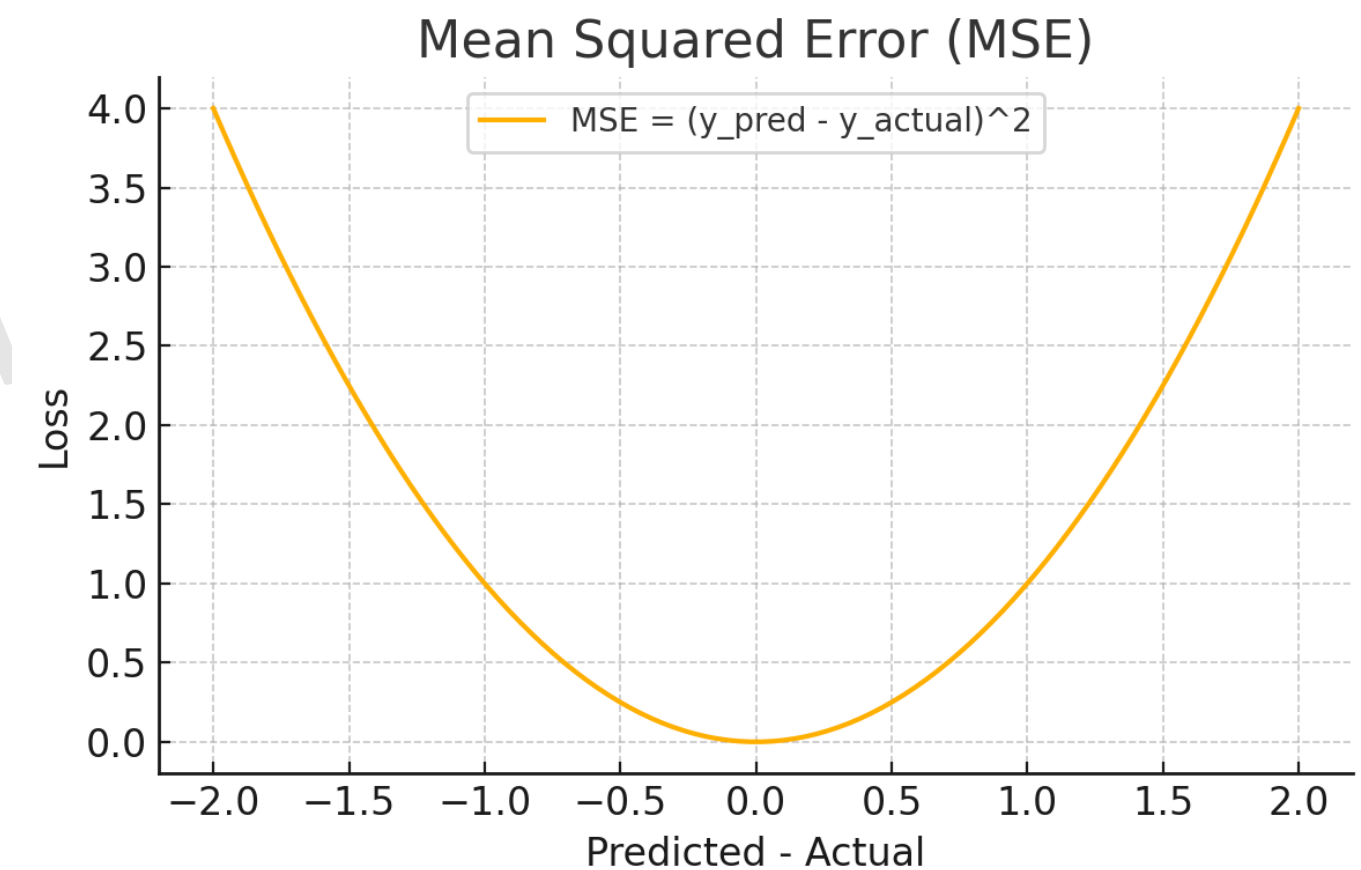
$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$



Para problemas de regresión, podemos utilizar la **función de pérdida L2 o MSE**, pero para problemas de clasificación, no resulta ser la opción correcta ya que observamos números binarios en la clasificación.

- ❖ **Ventaja:** Penaliza fuertemente los errores grandes, fácil de derivar.
- ❖ **Desventaja:** Sensible a valores atípicos (outliers).
- ❖ **Uso actual:** Regresión.
- ❖ **Fórmula:**

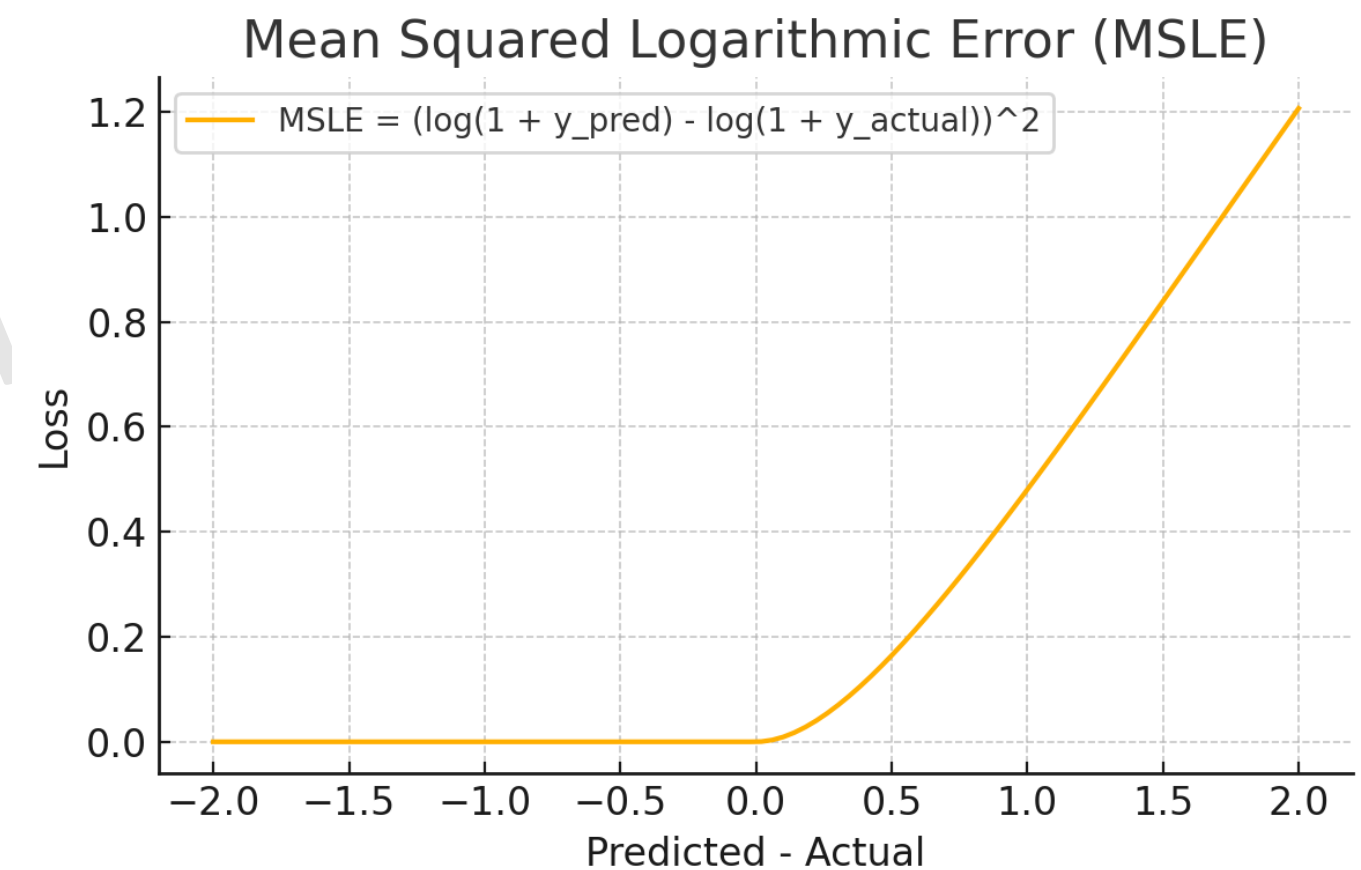
$$MSE = \frac{1}{n} \sum (y_i - \hat{y}_i)^2$$



La función **Mean Squared Logarithmic Error (MSLE)** mide la diferencia entre los logaritmos de las predicciones y los valores reales. Se utiliza principalmente en problemas de **regresión** cuando no se desea penalizar los grandes errores tanto como en el **Mean Squared Error (MSE)**.

- ❖ **Ventaja:** Es útil cuando el objetivo es penalizar más los errores relativos pequeños y menos los errores grandes. Útil cuando se trabaja con valores de salida que crecen exponencialmente.
- ❖ **Desventaja:** No funciona bien con valores cercanos a cero o negativos.
- ❖ **Uso actual:** Común en problemas de regresión donde los valores de salida varían en una amplia escala, como la predicción de precios o valores de crecimiento.
- ❖ **Fórmula:** Donde **y_i** es el valor real y **$\hat{y}_i$** es el valor predicho.

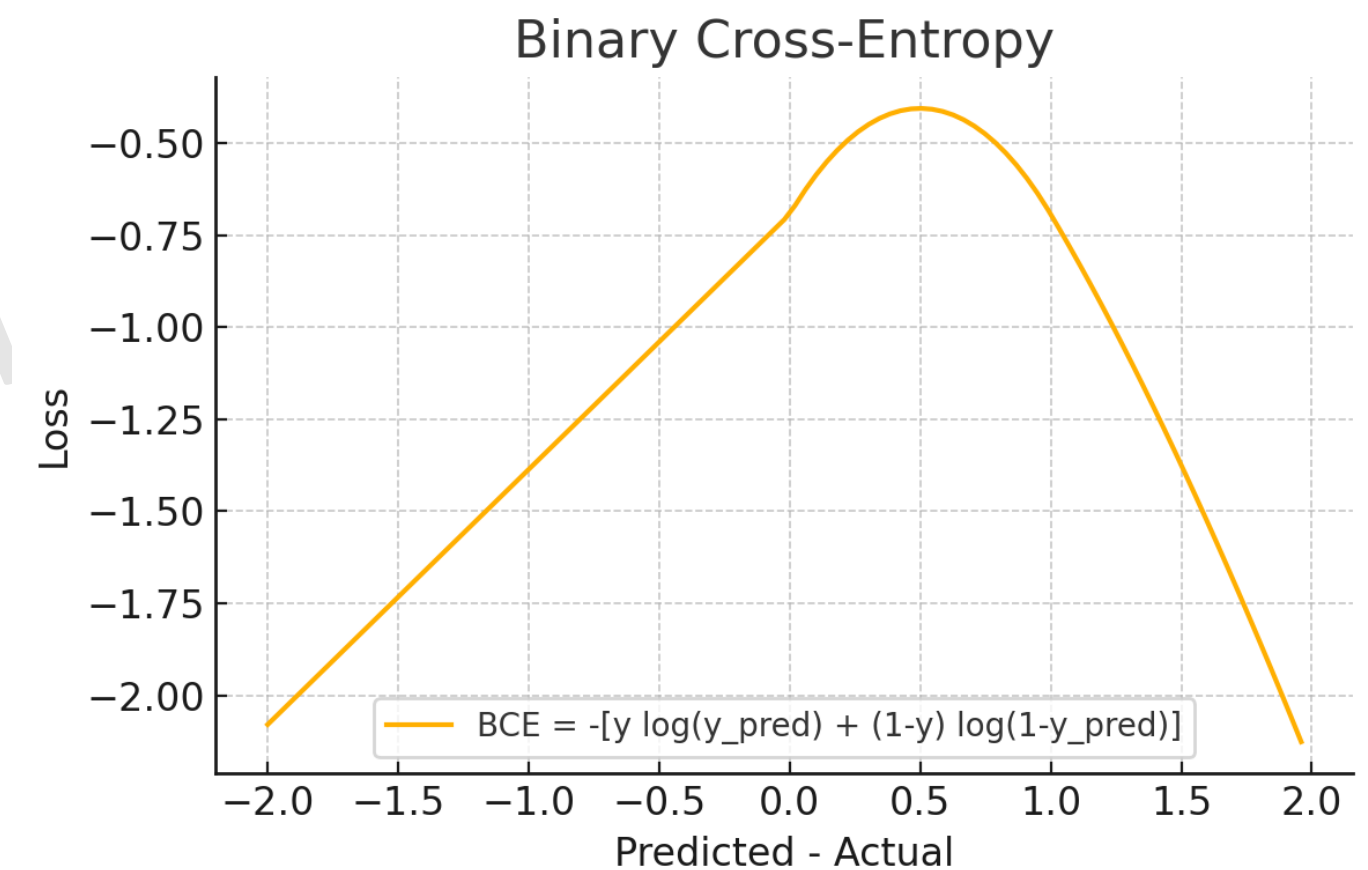
$$MSLE = \frac{1}{n} \sum_{i=1}^n (\log(1 + y_i) - \log(1 + \hat{y}_i))^2$$



Es una función de pérdida utilizada en problemas de **clasificación binaria**, es decir, cuando el objetivo es clasificar los datos en una de **dos clases** (por ejemplo, "positivo" o "negativo", "verdadero" o "falso"). Esta función mide la diferencia entre las probabilidades predichas por el modelo y las etiquetas reales.

- ❖ **Ventaja:** Convierte las salidas en probabilidades, adecuada para clasificación binaria.
- ❖ **Desventaja:** Las predicciones cercanas a 0 o 1 pueden causar inestabilidad.
- ❖ **Uso actual:** Clasificación binaria.
- ❖ **Fórmula:**

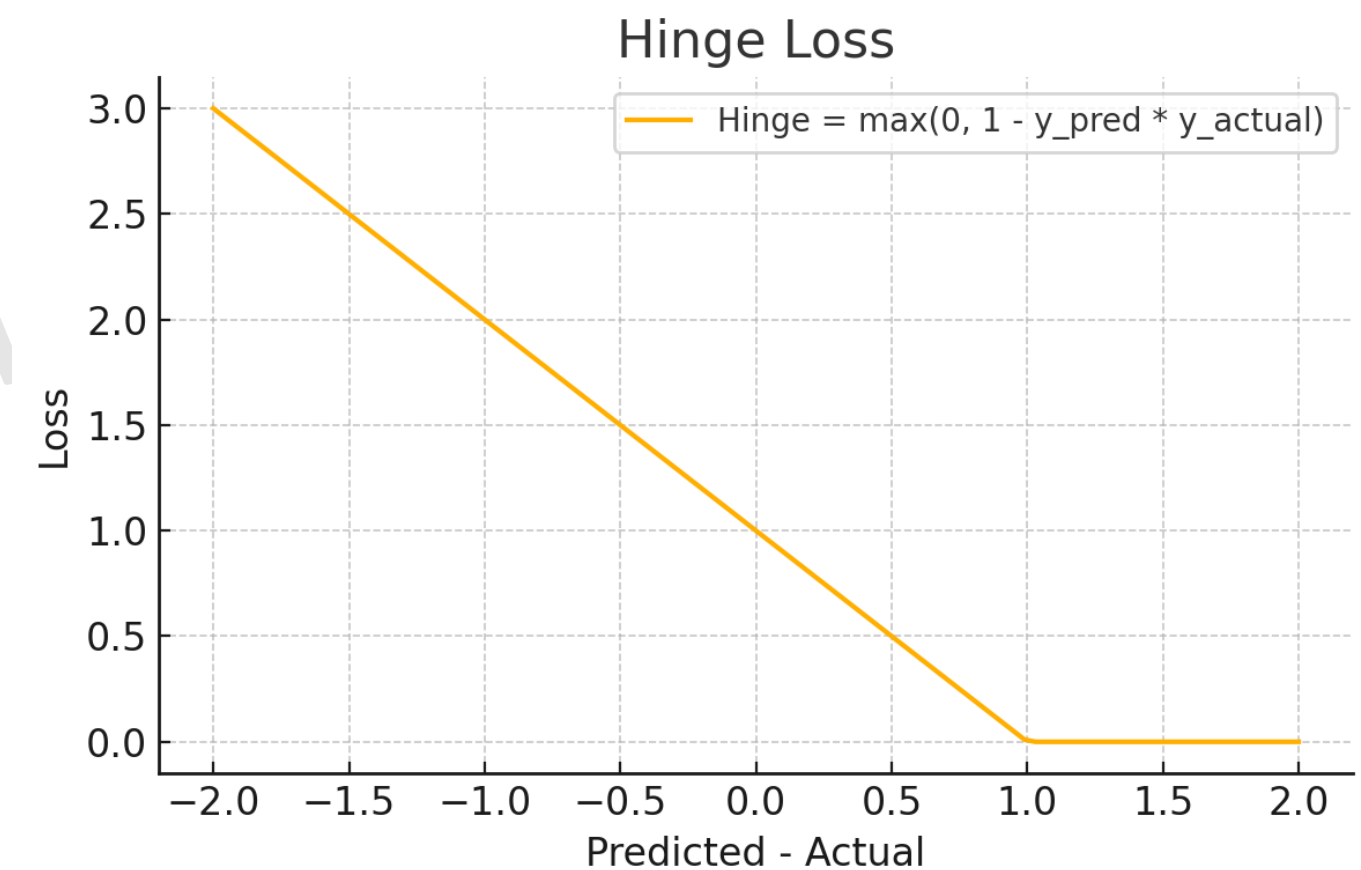
$$\text{Loss} = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$



Es una función de pérdida comúnmente utilizada para problemas de **clasificación binaria**, especialmente en **Máquinas de Soporte Vectorial (SVM)**. Se utiliza para **maximizar el margen** entre las clases, es decir, para asegurar que los ejemplos de diferentes clases estén lo más separados posible en el espacio de características

- ❖ **Ventaja:** Adecuada para problemas de clasificación binaria.
- ❖ **Desventaja:** Solo aplicable en problemas binarios, no aplicable a multiclase.
- ❖ **Uso actual:** Clasificación binaria en SVM (Support Vector Machines).
- ❖ **Fórmula:**

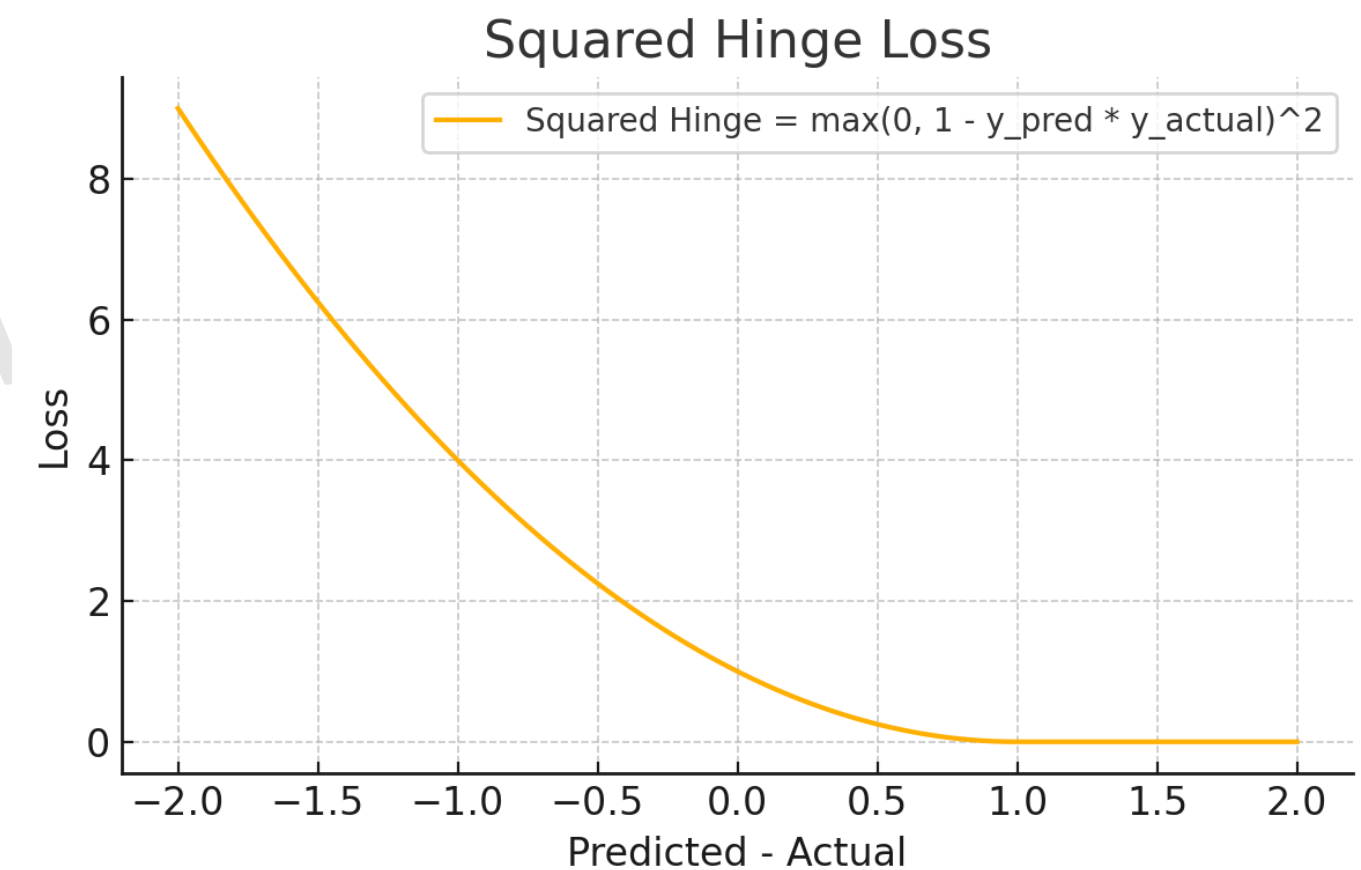
$$L = \max(0, 1 - y_i \cdot \hat{y}_i)$$



La **Squared Hinge Loss** es una variante de la **Hinge Loss**, utilizada en **Máquinas de Soporte Vectorial (SVM)** para clasificación. Penaliza más severamente las predicciones incorrectas o las que están cerca de la frontera de decisión.

- ❖ **Ventaja:** Ofrece una penalización más fuerte a las predicciones incorrectas que están cerca de la frontera de decisión en comparación con la Hinge Loss tradicional.
- ❖ **Desventaja:** Penaliza en exceso los puntos cercanos a la frontera, puede ralentizar el aprendizaje.
- ❖ **Uso actual:** Común en **máquinas de soporte vectorial (SVM)** y en tareas de clasificación binaria, donde se requiere una mayor separación entre las clases. Cuando se requiere una mayor penalización de los errores.
- ❖ **Fórmula:** Donde **y_i** es la etiqueta real (1 o -1) y **$\hat{y}_i$** es la predicción.

$$L = \frac{1}{n} \sum_{i=1}^n \max(0, 1 - y_i \cdot \hat{y}_i)^2$$



Utilizada en problemas de **clasificación multiclase** (más de dos clases). Similar a la entropía cruzada binaria, pero se aplica cuando hay varias categorías. Proporciona una métrica de qué tan bien predice el modelo la clase correcta entre varias opciones.

Aquí **y** es el valor real y **y-hat** es el valor predicho. Para los problemas de clasificación, los resultados posibles serán 1 o 0, por lo que los resultados reales y predichos serán 1 o 0. Si el valor real (y) es 1, la primera parte de la ecuación será cero. Si el valor real es 0, entonces la segunda parte de la ecuación será cero. Diferentes casos con diferentes valores de **y** y **y-hat** y resolvamos uno de ellos.

Caso 1: y = 1 y y-hat = 1

$$L(y, \hat{y}) = -(y_i \log(\hat{y}_i) + (1 - y_i)(\log(1 - \hat{y}_i)))$$

$$L(y, \hat{y}) = -(1 \log(1) + (1 - 1)(\log(1 - 1)))$$

$$L(y, \hat{y}) = -(1 * 0) + 0$$

$$L(y, \hat{y}) = 0$$

Caso 1: y = 1 y y-hat = 0

$$L(y, \hat{y}) = -(y_i \log(\hat{y}_i) + (1 - y_i)(\log(1 - \hat{y}_i)))$$

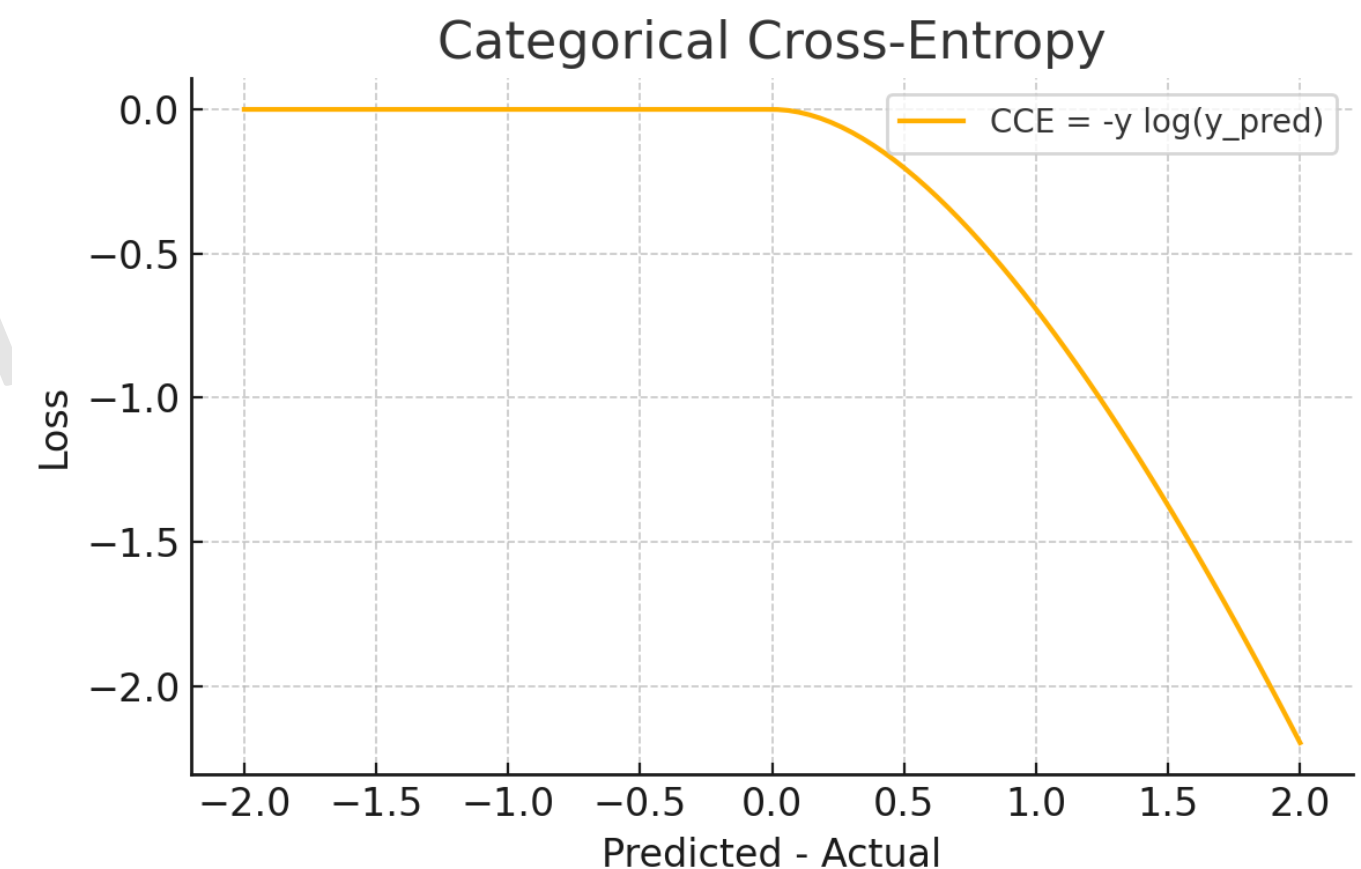
$$L(y, \hat{y}) = -(1 \log(0) + (1 - 1)(\log(1 - 0)))$$

$$L(y, \hat{y}) = -(1 * -\infty) + 0$$

$$L(y, \hat{y}) = \infty$$

- ❖ **Ventaja:** Mide probabilidades entre múltiples categorías.
- ❖ **Desventaja:** Puede ser problemática con clases desbalanceadas.
- ❖ **Uso actual:** Clasificación multiclase con etiquetas one-hot.
- ❖ **Fórmula:**

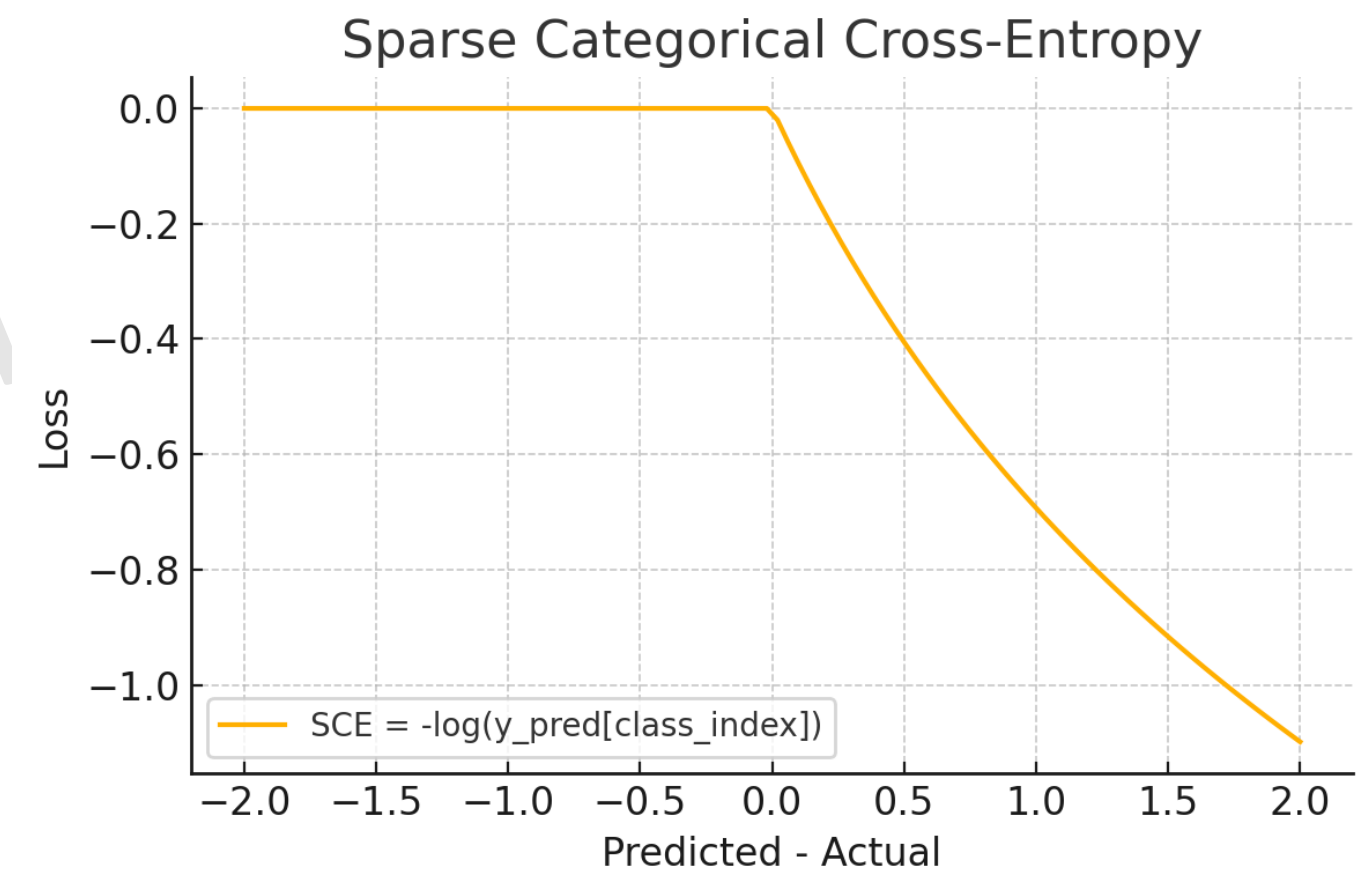
$$L(y, \hat{y}) = -(y_i \log(\hat{y}_i) + (1 - y_i)(\log(1 - \hat{y}_i)))$$



La **Sparse Categorical Cross Entropy** es una variante de la **Categorical Cross Entropy**, pero se usa cuando las etiquetas de clase están representadas como **enteros** en lugar de vectores one-hot (donde solo una clase es 1 y las demás son 0).

- ❖ **Ventaja:** Reduce el costo computacional en comparación con **Categorical Cross Entropy** cuando las etiquetas están en formato entero, ya que no es necesario convertir las etiquetas a un vector one-hot.
- ❖ **Desventaja:** Menos interpretable cuando se necesitan probabilidades por clase.
- ❖ **Uso actual:** Común en problemas de **clasificación multiclase** donde las etiquetas están en formato entero en lugar de vectores one-hot, como en redes neuronales profundas que resuelven problemas de clasificación de imágenes.
- ❖ **Fórmula:** Donde $\hat{y}_i$ es la probabilidad predicha para la clase verdadera y_i .

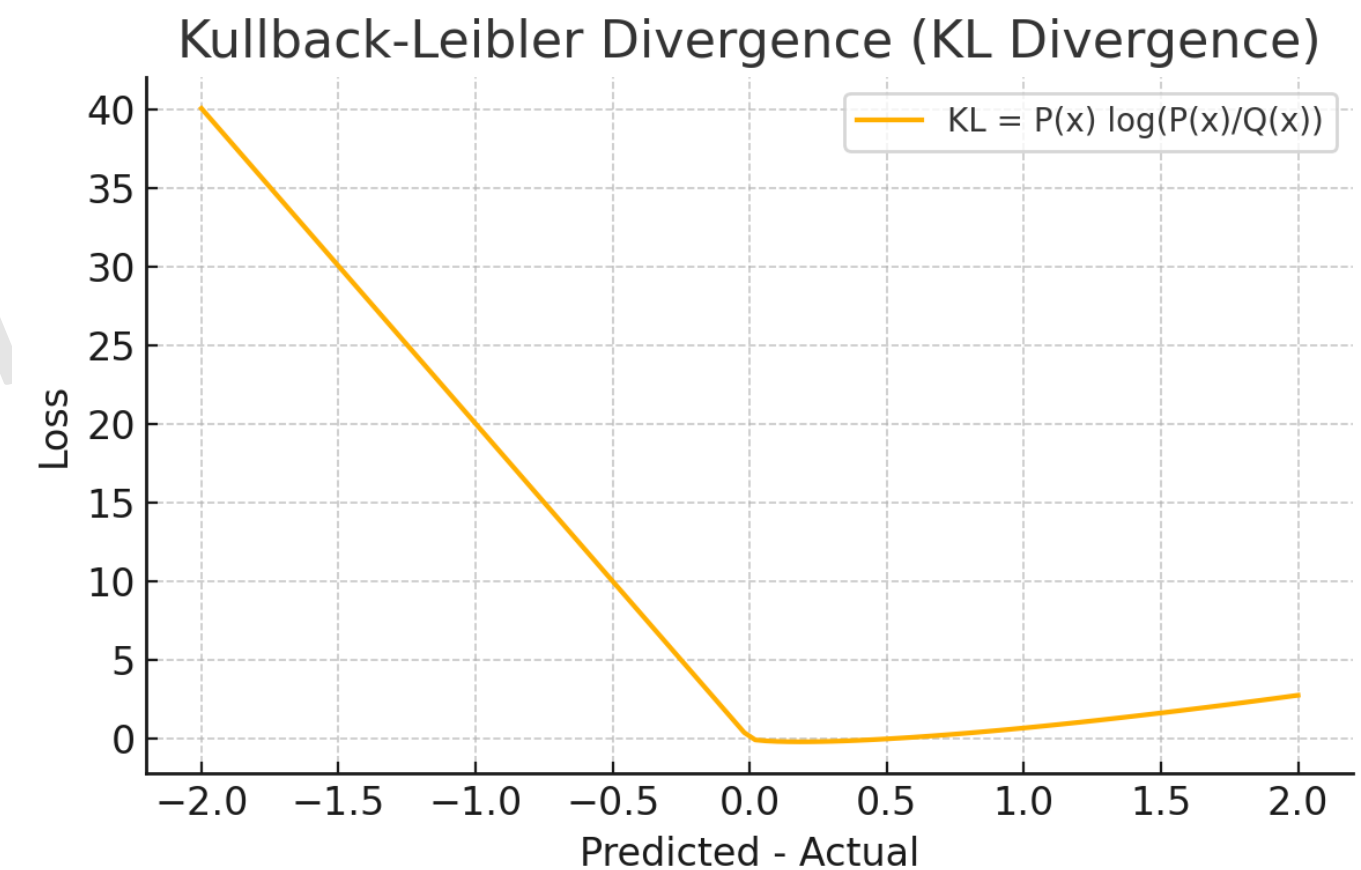
$$L = -\frac{1}{n} \sum_{i=1}^n \log(\hat{y}_{i,y_i})$$



La **Kullback-Leibler Divergence (KL Divergence)** mide la diferencia entre dos distribuciones de probabilidad: la distribución real y la distribución predicha. Es una métrica utilizada para medir cuán bien un modelo de predicción se ajusta a una distribución esperada.

- ❖ **Ventaja:** Es útil para medir la "distancia" entre dos distribuciones de probabilidad. Ideal para problemas donde el modelo debe aprender distribuciones de probabilidad complejas, como en **modelos generativos**.
- ❖ **Desventaja:** No es simétrica, puede ser inestable si las distribuciones contienen ceros.
- ❖ **Uso actual:** Se utiliza comúnmente en **modelos de autoencoders variacionales (VAE)** y otras técnicas de aprendizaje no supervisado, así como en tareas de predicción probabilística..
- ❖ **Fórmula:** Donde **P(xi)** es la distribución real y **Q(xi)** es la distribución predicha.

$$D_{KL}(P||Q) = \sum_{i=1}^n P(x_i) \log \frac{P(x_i)}{Q(x_i)}$$



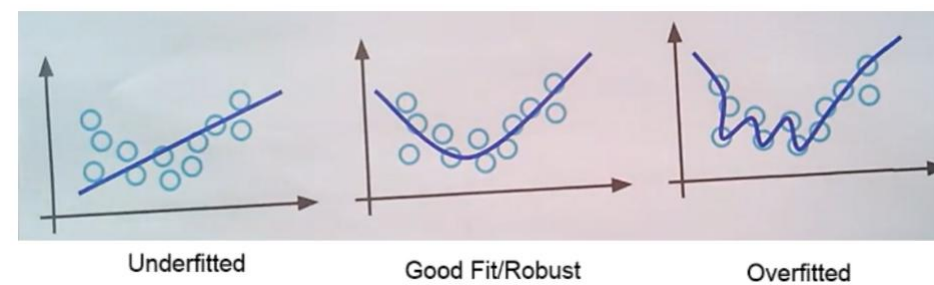
CONFIDENCIAL

Ejercicios prácticos

Estudiantes

Regularización

Los datos contienen información y ruido. Teniendo en cuenta la imagen a continuación, la primera imagen es un ejemplo de subajuste, la segunda imagen es un buen ajuste y la tercera imagen contiene muchos parámetros y es más compleja, lo que indica que el modelo se sobreajustará.



Underfit: este modelo no es información capturada ni ruido.

Good fit: este modelo ha capturado solo la información.

Overfit: este modelo ha capturado tanto información como ruido.

El overfit es una situación en la que un modelo ofrece un buen rendimiento en el conjunto de entrenamiento, pero un rendimiento deficiente en el conjunto de prueba. El underfit es una situación en la que el rendimiento del modelo es deficiente tanto en el conjunto de entrenamiento como en el de prueba. El término "regularización" describe métodos para calibrar modelos de ML/DL para ajustar la función de pérdida y evitar el overfit o el underfit.

Cuanto más complejo es un modelo, más parámetros debe ajustar y el modelo tiende a overfit. Las redes neuronales son propensas al overfitting debido a la mayor cantidad de parámetros. Las redes neuronales pueden modelar funciones complejas y de orden superior, lo que las hace más propensas al sobreajuste. La forma de obligar a una red neuronal a no sobreajustarse es reducir la complejidad de la red.

La regularización **L1 y L2** son técnicas de uso común en el aprendizaje automático tanto para la regresión como para la clasificación.

Habrán muchos pesos en una red neuronal, algunos de los cuales influirán en que la función de pérdida sea alta. El objetivo es reducir o eliminar dichos pesos agregando un término de penalización en la función de pérdida.

- ❖ **Regularización L1 o Lasso:** toma la suma de los valores de peso absolutos. Lasso también se conoce como el operador de contracción y el operador de selección menos absolutos. Debido a que utiliza pesos absolutos, algunos pesos que están cerca de cero se convertirán en cero y se eliminarán, lo que implica que la conexión entre los dos nodos se desconectará, lo que reducirá la complejidad.
- ❖ **Regularización L2 o Ridge:** toma la suma del cuadrado de los pesos. El método Ridge no hará que los pesos sean cero; en cambio, reducirá significativamente el valor de los pesos. Las conexiones entre nodos con información menos significativa para la predicción tendrán menos influencia, lo que reduce la complejidad.
- ❖ **Data Augmentation:** El aumento de datos es otra técnica utilizada en redes neuronales para reducir el problema de subajuste. Es similar a la técnica de sobremuestreo utilizada en el aprendizaje automático. Al agregar versiones ligeramente modificadas de datos ya existentes o generar nuevos datos sintéticos a partir de datos existentes, se utiliza el aumento de datos para expandir la cantidad de datos disponibles. Cuando se entrena un modelo de aprendizaje profundo, funciona como regularizador y ayuda a reducir el subajuste. El método de aumento de datos se aplica a los datos de imagen.



Regularización - Dropout

¿Por qué necesitamos Dropout?

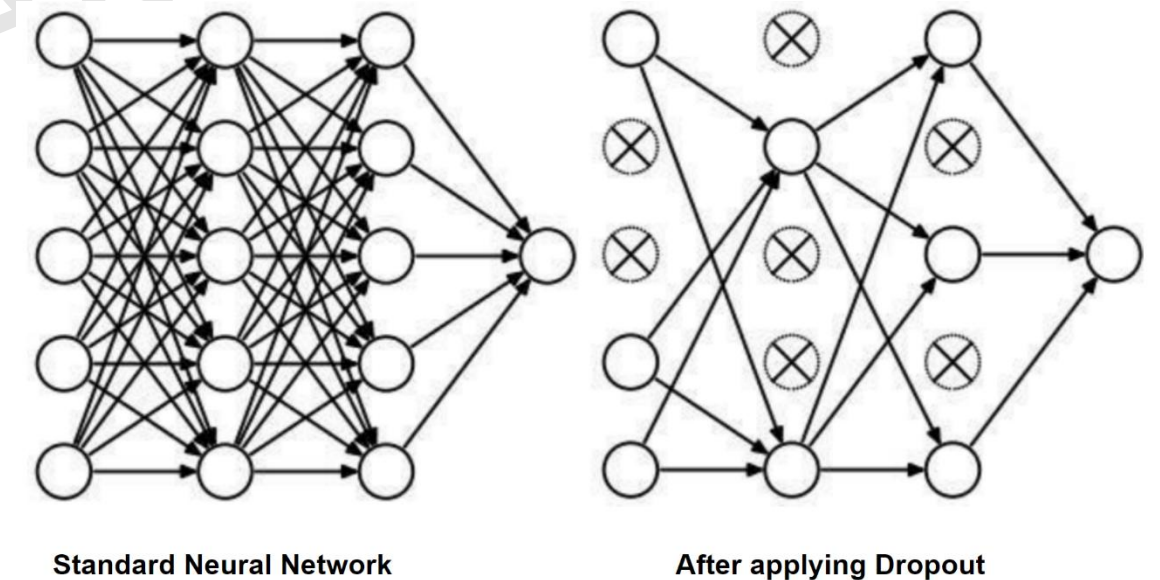
Una capa completamente conectada ocupa la mayoría de los parámetros. Por lo tanto, las neuronas desarrollan codependencia entre sí durante el entrenamiento, lo que limita la potencia individual de cada neurona, lo que lleva a un sobreajuste de los datos de entrenamiento. Dropout es un método de regularización en redes neuronales que ayuda a reducir el aprendizaje interdependiente entre neuronas.

¿Cómo funciona Dropout?

Supongamos que un equipo ha sido entrenado para competir en un evento. Es bien sabido que los diferentes participantes tienen diferentes capacidades y que cada contribución individual para ganar el evento es única. Algunos pueden hacer contribuciones significativas, mientras que otros no. Una situación similar ocurre en las redes neuronales, donde algunos nodos son excelentes predictores mientras que otros no. Los que son malos predictores agregan ruido y necesitamos un método para entrenar a todos los importantes.

En lugar de entrenar a todo el equipo, podemos seleccionar algunos miembros del equipo y entrenarlos, y en la siguiente ronda, podemos seleccionar otro grupo de miembros. De esta manera, podremos identificar el potencial de cada miembro del equipo.

Considerando un ejemplo similar en la red neuronal, en cada capa, seleccionamos algunos nodos aleatorios (digamos $p = 0,5$, es decir, el 50% de los nodos (neuronas)) para el entrenamiento y descartamos los nodos restantes. En la segunda vez, seleccionamos otro 50% aleatorio de los nodos y descartamos el resto, y así sucesivamente. Podemos pensar en esto como aprendizaje en conjunto también. De esta manera, cada nodo en la red se entrena para diferentes nodos en la conexión



Regularización – Batch Normalization

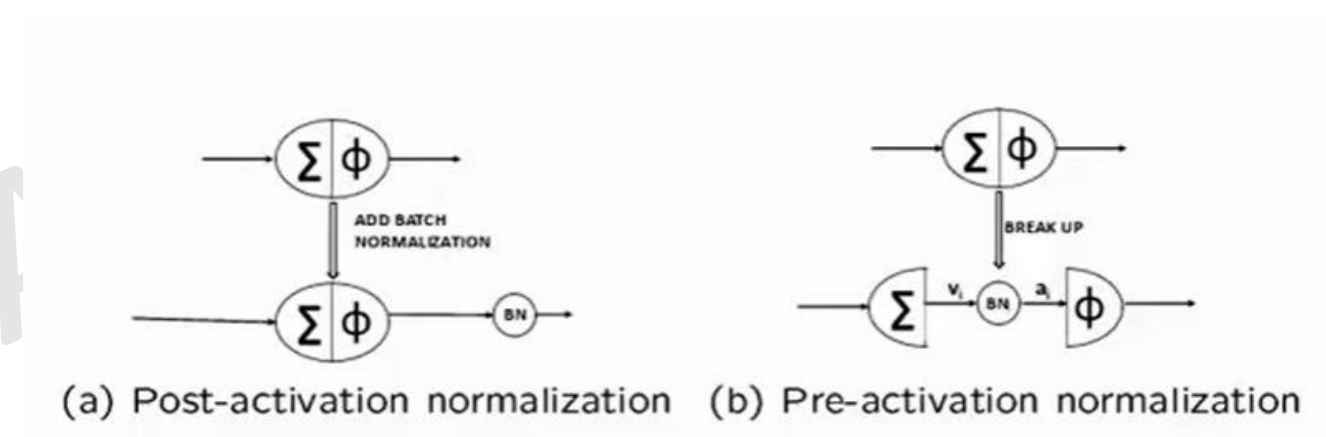
La normalización por lotes es una de las técnicas de regularización más potentes que se utilizan para superar el problema del sobreajuste en las redes neuronales.

La normalización por lotes es una técnica para mejorar el rendimiento y la estabilidad de las redes neuronales. La normalización por lotes normalizará las entradas de cada capa de tal manera que tengan una activación de salida media de cero y una desviación estándar de uno.

Por lo general, hay dos etapas en las que se aplica la normalización por lotes:

1. Antes de la función de activación (no linealidad)
2. Después de la función de activación (no linealidad).

La normalización por lotes normalmente se utiliza después de las funciones de activación.



Φ - Activation Function
 Σ - Summation of Inputs and Weights
BN - Batch Normalization

CONFIDENCIAL

Preguntas